Proof Path Theory

Certificates, Compression, Discovery Fields, and Mathematical Memory

Komorebi P.

Version 1.2.0 / 18 October 2026

Abstract

A finished proof is a certificate of implication and a compressed projection of a larger mathematical object. The larger object may contain alternative derivations, examples, failed routes, exact computations, boundary cases, provenance, and bridges to later questions. This paper develops a finite Proof Path Theory that keeps these layers distinct. A proof field is a typed acyclic justification hypergraph. Certificates are selected certified subfields that derive declared targets from admissible leaves. In the fixed AND-DAG case, dependency ancestry gives a unique minimal certificate. With alternative justifications, minimum certificate selection is NP-complete by reduction from Set Cover.

Compression is defined relative to task families, producing endpoint, audit, and discovery equivalence, a recoverability preorder, vector-valued loss, and expected recovery debt. A latent bridge graph models declared future reach. Its weighted coverage utility is monotone and submodular, yielding a greedy cardinality guarantee. The framework also treats versioned proof objects, evidence ledgers, correlated review, audit queues, and goal-dependent refinement. The accompanying Python package implements the finite theory, validates proof fields, extracts certificates, computes retention policies, and reproduces the examples. The theory preserves deductive standards while formalizing what compression retains, discards, and makes costly to reconstruct. Version 1.2 adds a Lean-checked bridge from declared proof fields to coherent realization spaces, with whole-support gluing, scoped countermodels, linear obstruction alternatives, exact finite-field observation counts, and a native declaration export.

Status of the work. The mathematical theorems below concern finite declared proof fields, task families, bridge graphs, covariance matrices, and queue models. Their proofs are given in the text. The computational results are exact finite checks or deterministic numerical examples generated from stated inputs. No simulation is presented as an empirical law of mathematical discovery. Version 1.2 kernel-checks the stated realization bridge and selected general linear and compression results. This is not a formalization of every result in the paper or empirical proof of social distortion. Historical priority and comparison with neighboring literatures remain separate questions.

Contents

1 Introduction **4**

2 Claim types and logical boundaries **5**

3 Typed proof fields **6**

4 Certificates and verification closure **7**
4.1 The fixed AND-DAG case 8

5 Alternative derivations and certificate complexity **9**

6 Certified paths as a category **10**

7 Compression relative to a task family **11**
7.1 Pure retention operators 12

8 Endpoint, audit, and discovery equivalence **14**

9 Vector-valued loss **15**

10 Compression debt **16**

11 A proof ecology rather than one ideal form **16**

12 Proof compilers and shared ledgers **17**

13 Latent bridge graphs **17**
13.1 Bridge criticality and dominators 18

14 Retention under a budget **19**

15 Versioned proof objects **20**

16 Evidence ledgers and provenance **21**

17 Correlated review **22**

18 Verification bandwidth and audit queues **24**

19 Recursive proof cubes and expansion depth **25**

20 Worked finite examples **26**
20.1 A fixed AND-DAG 26
20.2 Two certified proofs 26
20.3 Set Cover encoded as certificate selection 27
20.4 Bridge retention 27
20.5 Correlated review and audit capacity 27

21 Reference implementation **27**
21.1 Command-line interface . . . 28
21.2 JSON contract . . . 29
21.3 Tests and build discipline . . . 29

22 The ambient proof field and the meaning of failure **30**

23 Komorebi Space, realization fibers, and informative questions **31**
23.1 Exact finite realization and obstruction certificates . . . 32
23.2 The information gained by a new observation . . . 33

24 A checked realization bridge for proof fields **34**
24.1 The declared rule system and its models . . . 34
24.2 From patches to a contravariant view system . . . 35
24.3 The correct coverage condition is a whole support . . . 35
24.4 Protected realization over a field . . . 36
24.5 Counting what remains after an observation . . . 37
24.6 Executable witnesses and a native declaration export . . . 38
24.7 Verification scope and reproducibility . . . 38

25 Shared interpretation, review dependence, and identification **38**
25.1 Correlated noise and persistent common bias . . . 39
25.2 When an identity effect can actually be measured . . . 39
25.3 What the new bridge does and does not claim . . . 40

26 What the theory establishes **40**

27 Limitations and claim boundaries **41**

28 Open theorem program **41**

29 Conclusion **42**

A Theorem ledger **43**

B Reproduction **44**

C Canonical proof-field schema excerpt **45**

1 Introduction

Mathematics usually publishes a proof after the proof has been compressed. Definitions have stabilized. Failed attempts have disappeared. A long calculation has become a lemma. A representation change that made the argument visible is replaced by a direct sentence. This is often an improvement. The final certificate is easier to verify and easier to reuse.

Compression can also remove mathematical information that is not needed for the present theorem. An example may explain why a definition was chosen. A failed route may isolate the exact boundary of a method. An alternative proof may carry a bridge to another field. A computation may reveal a stronger statement than the one originally sought. None of these objects is automatically part of the final implication

$$A \vdash B.$$

They may still be part of the mathematical event that produced it.

Proof Path Theory studies this larger object without changing the standard of proof. Its central separation is

$$\boxed{\text{certificate of validity} \quad \neq \quad \text{expanded discovery field.}}$$

The certificate answers whether the conclusion follows. The discovery field records how the conclusion became reachable, what alternatives were considered, what was lost during compression, and which omitted structures may matter later.

The finite theory developed here has six connected layers.

1. A typed proof field records claims, admissible leaves, justifications, evidence, and targets.
2. A certified subfield separates proved or otherwise certified transitions from candidates and interpretations.
3. A certificate is a selected subfield that derives the target.
4. Compression is evaluated relative to declared tasks rather than by length alone.
5. A bridge graph represents possible future reach and makes archival retention an optimization problem.
6. Versioning, review, audit, and refinement describe the life of the proof after the first certificate exists.

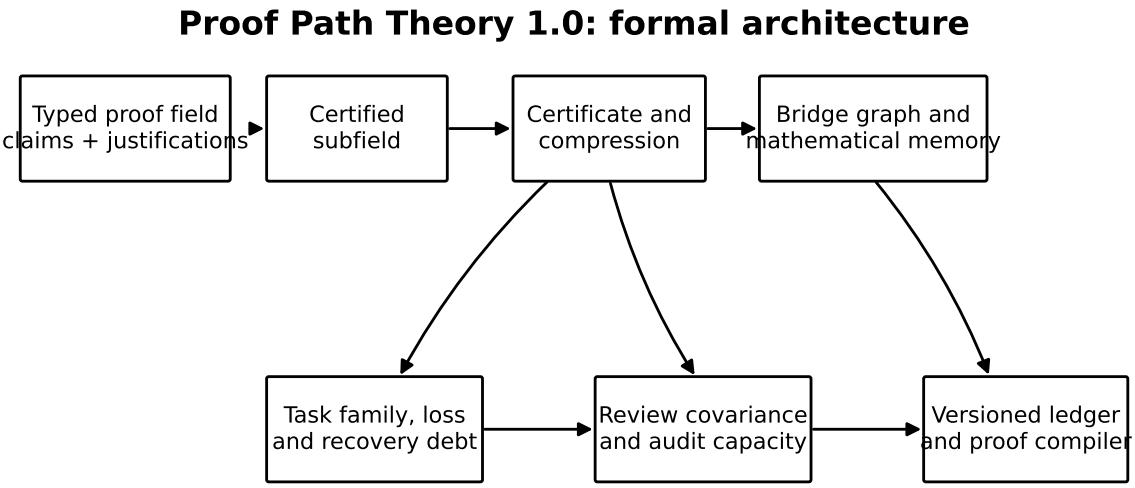


Figure 1: The formal architecture. The certificate is one verified projection of a larger versioned proof field.

The ambient development treats the proof field as an ambient field of candidate and certified paths. It connects coherent local proof views to the strict Komorebi realization construction, and adds an exact finite information measure for informative observations. The mathematical validity of a proof remains distinct from how communities discover, interpret, or review it.

The theory is intentionally finite at its foundation. Finite objects make every closure condition and algorithm explicit. Infinite, probabilistic, and higher categorical extensions are listed later as open programs rather than silently assumed.

2 Claim types and logical boundaries

A useful theory of proof paths must not turn every recorded object into a proof step. A notebook may contain a theorem, a conjecture, a failed analogy, a numerical experiment, and a historical interpretation on the same page. Their coexistence does not give them the same logical status.

Definition 2.1 (Evidence kind)**.** An evidence kind is a declared type attached to a record supporting a claim or justification. The core finite vocabulary used here is

formal proof, assumption, exact computation,
numerical evidence, conjecture, external result,
counterexample, interpretation.

An evidence record also states its source, scope, and whether it certifies the justification to which it is attached.

The word “certifies” is deliberately separate from the evidence label. An exact computation may certify

a finite identity while providing only numerical evidence for a general statement. An external result may certify a step only on the domain covered by its hypotheses. The scope field is therefore part of the mathematical object.

Warning 2.2 (Evidence types are not one ranking). The evidence kinds do not form a universal total order. A formal proof of a weak statement and an exact computation of a stronger finite instance cannot be compared by one scalar without specifying the task. The ledger retains type and scope instead of replacing them with a single confidence number.

This separation will be used throughout. Candidate edges may remain in the discovery field, but only certified edges enter a proof certificate.

3 Typed proof fields

The first formal object is a directed hypergraph. A justification may require several premises at once, and a claim may admit several alternative justifications.

Definition 3.1 (Typed proof field)**.** A finite typed proof field is a tuple

$$\mathcal{P} = (V, \mathcal{J}, A, T, \tau, \mathcal{E}, \mathbf{v}, \mathbf{c}),$$

where:

- V is a finite set of claim nodes.
- $\mathcal{J}$ is a finite set of justifications.
- Each $j \in \mathcal{J}$ has a finite premise set $\mathrm{prem}(j) \subseteq V$ and one conclusion $\mathrm{con}(j) \in V$.
- $A \subseteq V$ is the set of required premises of the declared judgment.
- $T \subseteq V$ is the nonempty target set.
- τ assigns node and justification types.
- $\mathcal{E}$ is a typed evidence ledger.
- $\mathbf{v}$ assigns nonnegative value coordinates to nodes.
- $\mathbf{c}$ assigns nonnegative storage, audit, recovery, and cognitive costs.

The dependency graph contains an edge $u \to v$ whenever $u \in \mathrm{prem}(j)$ and $v = \mathrm{con}(j)$ for some justification j. It is required to be acyclic.

A single justification is conjunctive in its premise set. All listed premises must be available. Multiple justifications with the same conclusion are alternatives. This gives the familiar AND/OR structure of proof search without forcing it into an ordinary graph.

Definition 3.2 (Admissible leaf)**.** A node is an admissible leaf when it is a required premise, a declared axiom, or an explicitly cited external result. A zero-premise certified justification may also introduce a leaf when the proof system permits it.

Definition 3.3 (Certified subfield)**.** The certified subfield $\mathcal{P}^{\mathrm{cert}}$ contains every node of $\mathcal{P}$ and only those justifications whose evidence and scope have been declared sufficient to certify the recorded transition.

Definition 3.4 (Derivability)**.** Let $K \subseteq V$ be a selected node set and let $H \subseteq \mathcal{J}$ be a selected justification set. The derivable set $D(K, H)$ is the least subset of K satisfying:

(i) every selected admissible leaf belongs to $D(K, H)$;

(ii) if $j \in H$, $\mathrm{prem}(j) \subseteq D(K, H)$, and $\mathrm{con}(j) \in K$, then $\mathrm{con}(j) \in D(K, H)$.

When only certified justifications are allowed, write $D_{\mathrm{cert}}(K, H)$.

Proposition 3.5 (Finite derivability)**.** *For a finite acyclic proof field, $D(K, H)$ is uniquely determined and can be computed in one topological pass through the dependency graph.*

Proof. Acyclicity gives a topological ordering of the nodes. Begin with the selected admissible leaves. At each later node, test whether one selected incoming justification has all of its premises already derived. Add the node exactly when such a justification exists. No later step can change the status of an earlier node because all dependencies point forward. The resulting set satisfies the closure rules. Any set satisfying the rules must contain every node added by this pass, so the result is the least such set and is unique. □

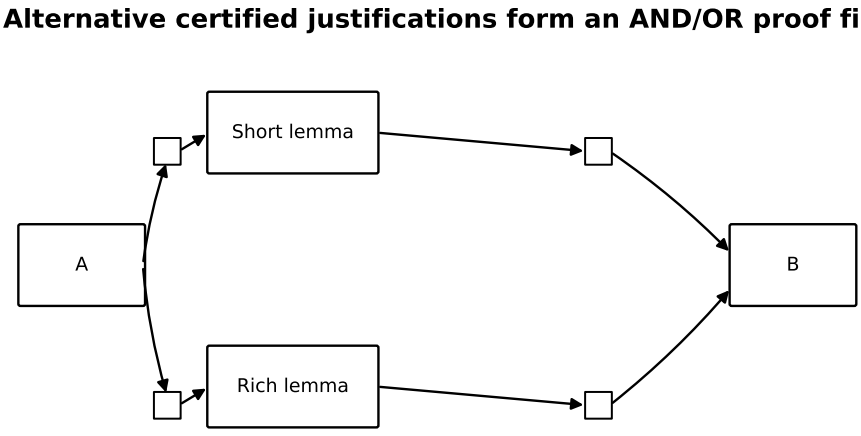


Figure 2: A proof field with alternative certified justifications. Squares represent justification records.

4 Certificates and verification closure

Definition 4.1 (Certificate)**.** A certificate for a requested target set $T_0 \subseteq V$ is a pair

$$\mathcal{C} = (K, H), \qquad K \subseteq V, \qquad H \subseteq \mathcal{J}^{\mathrm{cert}},$$

such that

$$A \cup T_0 \subseteq K \qquad \text{and} \qquad T_0 \subseteq D_{\mathrm{cert}}(K, H).$$

The certificate is node-minimal, justification-minimal, or cost-minimal according to the declared objective.

The canonical target request is usually a subset of the declared set T, although the same object can certify an intermediate node selected for local analysis. This definition preserves the requested endpoint judgment and requires every target to be reconstructed from certified transitions. Extra nodes may remain in a certificate, but they do not help minimality under nonnegative costs.

4.1 The fixed AND-DAG case

The simplest case occurs when every relevant derived node has exactly one certified incoming justification. Then every recorded dependency is mandatory.

Definition 4.2 (AND-DAG)**.** A certified proof field is an AND-DAG relative to target set T_0 when every nonleaf node in the certified ancestry of T_0 has exactly one certified incoming justification.

Let $\mathrm{Anc}(T_0)$ denote the node ancestry obtained by following the unique certified justifications backward from T_0.

Theorem 4.3 (Unique verification core)**.** *Let $\mathcal{P}$ be an AND-DAG relative to T_0. The selected nodes*

$$K_{\mathrm{core}} = A \cup T_0 \cup \mathrm{Anc}(T_0)$$

together with the unique incoming justification of every selected nonleaf node form a certificate. This certificate is the unique node-minimal and justification-minimal certificate for the recorded derivation.

Proof. Backward traversal from each target selects the unique justification that can derive it and every premise required by that justification. Repeating the process terminates because the dependency graph is finite and acyclic. The selected leaves are admissible, and a forward topological pass derives every selected node, so the construction is a certificate.

Let (K, H) be any certificate for the same target set. Each target must be derived. Since it has one certified incoming justification, that justification must lie in H, and all of its premises must lie in K. Applying the same argument recursively forces every node and justification in the backward ancestry into (K, H). Therefore every certificate contains the constructed core. Uniqueness and minimality follow. □

Corollary 4.4 (Linear extraction)**.** *The unique core of an AND-DAG can be extracted in time linear in the size of the relevant certified ancestry, assuming indexed adjacency and constant-time membership bookkeeping.*

This bound concerns the indexed graph algorithm. It is not a verified running-time bound for every scanning routine in the reference Python package.

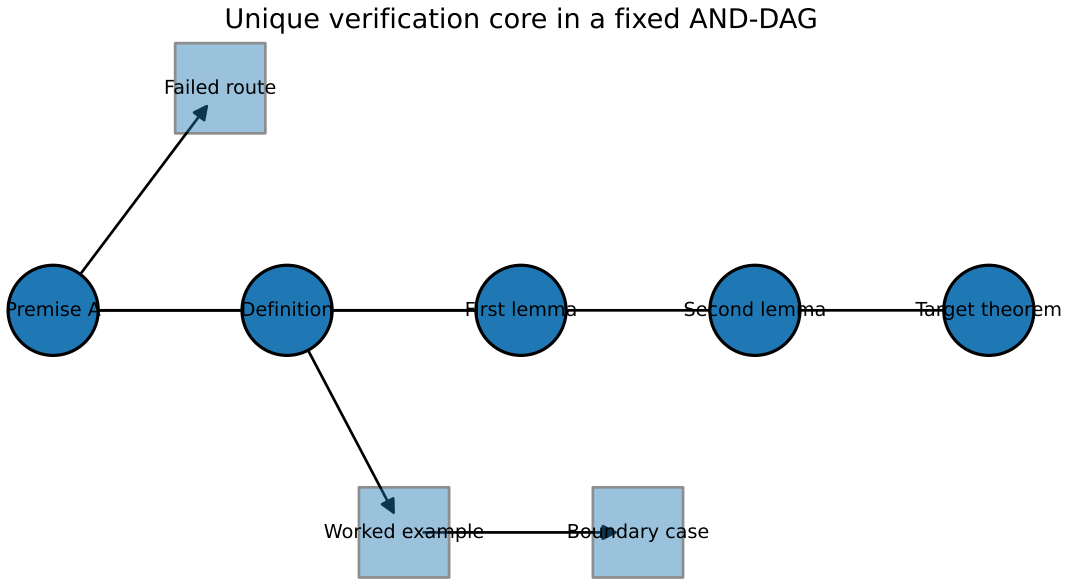


Figure 3: The unique verification core and optional discovery material in a fixed AND-DAG.

Remark 4.5. The theorem identifies the minimum certificate for the recorded derivation. It does not claim that the derivation is globally shortest among all possible proofs of the theorem. A different proof field may have a smaller core.

5 Alternative derivations and certificate complexity

When a claim admits several certified justifications, certificate extraction is no longer a backward closure problem. A local choice may be reused by several later obligations, which couples decisions across the field.

Problem 5.1 (Minimum Certificate Decision)**.** Given a finite acyclic certified proof field, a target set, nonnegative integer node and justification costs, and a budget B, determine whether a certificate of total cost at most B exists.

Theorem 5.2 (Minimum certificate selection is NP-complete)**.** *Minimum Certificate Decision is NP-complete, even when every nonzero cost is a unit node cost and all justification costs are zero.*

Proof. The problem belongs to NP. A proposed node and justification set can be checked in polynomial time by verifying references, computing derivability in topological order, and summing costs.

For NP-hardness, reduce Set Cover. Let $U = \{u_1, \dots, u_m\}$ be the universe, let $S_1, \dots, S_n \subseteq U$ be the available sets, and let k be the cover budget. Construct one optional admissible leaf node s_i for each set S_i, with unit cost. Construct one zero-cost obligation node e_j for each element u_j. Whenever $u_j \in S_i$, add a certified justification

$$\{s_i\} \Longrightarrow e_j.$$

Add a zero-cost target node t with the certified justification

$$\{e_1, \ldots, e_m\} \Longrightarrow t.$$

The dependency graph is acyclic. A certificate for t must derive every e_j. Deriving e_j requires at least one selected source node s_i with $u_j \in S_i$. One selected s_i may discharge several obligations, and its cost is paid once. Therefore a certificate of cost at most k exists exactly when the original Set Cover instance has a cover of size at most k. The reduction is polynomial. □

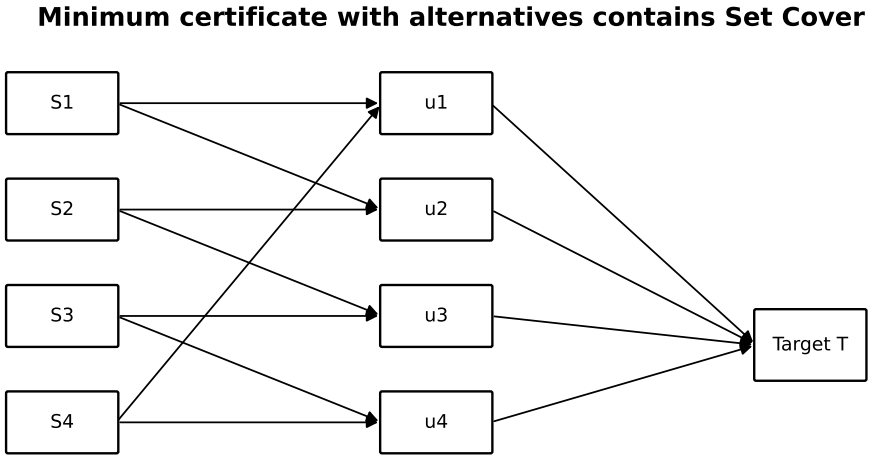


Figure 4: The Set Cover encoding used in the complexity proof.

Remark 5.3 (Algorithmic consequence). Exact minimum certificate search is practical for small fields or fields with few alternatives. Large formal libraries require branch and bound, dynamic programming on restricted structures, integer programming, approximation, or proof-system-specific methods. The reference package includes an exact finite search with an explicit state limit. It does not hide the exponential worst case.

6 Certified paths as a category

The hypergraph model records local justifications. A complementary view treats complete certified transformations as morphisms.

Definition 6.1 (Certified proof category)**.** A certified proof category Cert has mathematical states as objects and certified transformations as morphisms. Identity morphisms retain a state unchanged. Composition is sequential use of compatible certified transformations.

A state may be a proposition, a diagram, an exact finite computation, a coordinate representation, or a structured collection of claims. Candidate analogies may live in a larger discovery graph, but they do not enter Cert until their logical force and domain have been certified.

Proposition 6.2 (Functorial transport)**.** *Let $F : \mathsf{Cert}_1 \to \mathsf{Cert}_2$ be a functor. If*

$$A \xrightarrow{f_1} S_1 \xrightarrow{f_2} \cdots \xrightarrow{f_r} B$$

is a certified path in Cert_1, then

$$F(A) \xrightarrow{F(f_1)} F(S_1) \xrightarrow{F(f_2)} \cdots \xrightarrow{F(f_r)} F(B)$$

is a certified path in Cert_2.

Proof. A functor maps objects to objects and morphisms to morphisms. It preserves identities and composition. The image of the composite path is therefore the composite of the image morphisms, which is certified in the target category. □

The proposition is elementary, but its interpretation is central. A proof may be transported into another language without changing validity. What changes is visibility. A spectral representation may expose a symmetry hidden in physical coordinates. A finite exact computation may expose the correct statement before a general derivation is known. A formal proof assistant may expand dependencies that a journal proof compresses.

Definition 6.3 (Compositional certificate map)**.** A compositional certificate map is a functor

$$\mathcal{C} : \mathsf{Expanded} \longrightarrow \mathsf{Certificate}$$

that sends expanded certified paths to checkable certificate paths and preserves identity and composition.

Proposition 6.4 (Local audit composes)**.** *If every primitive morphism in a finite path has an auditable certificate and the certificate map is compositional, then the composite path is auditable from the primitive certificates and the declared composition rules.*

Proof. Proceed by induction on path length. The length-zero case is an identity. Assume the first r morphisms have an auditable composite certificate and append one certified morphism. Functoriality identifies the new certificate with the composition of the earlier certificate and the primitive certificate of the appended morphism. The declared composition rule completes the audit. □

7 Compression relative to a task family

A representation is not simply good or bad. It is sufficient or insufficient for a declared family of questions.

Definition 7.1 (Task)**.** A task q is a query whose answer is defined on the full proof field. Examples include:

- verify the target theorem;
- reconstruct every primitive dependency;
- teach the conceptual bottleneck;
- recover a boundary example;

- identify all claims affected by a revision;
- search for bridges to a future target family.

A task family is a set $\mathcal{Q}$ of such queries.

Definition 7.2 (Task-sufficient representation)**.** A representation R of a proof field $\mathcal{P}$ is sufficient for task q when there exists a decoder d_q such that

$$q(\mathcal{P}) = d_q(R).$$

It is $\mathcal{Q}$-sufficient when it is sufficient for every $q \in \mathcal{Q}$.

This definition includes exact reconstruction as one task, but it does not require exact reconstruction for every purpose. A certificate can be sufficient for verification and insufficient for teaching. A discovery archive can be sufficient for bridge search and unnecessarily large for routine checking.

Theorem 7.3 (Task information loss criterion)**.** *Let C be a compression map and let q be a task. If there exist proof fields $\mathcal{P}_1$ and $\mathcal{P}_2$ such that*

$$C(\mathcal{P}_1) = C(\mathcal{P}_2) \qquad \textit{but} \qquad q(\mathcal{P}_1) \neq q(\mathcal{P}_2),$$

then the compressed representation is not sufficient for q.

Proof. Suppose a decoder d_q existed. Applying it to the common compressed representation would have to produce both $q(\mathcal{P}_1)$ and $q(\mathcal{P}_2)$. Since these answers differ, no such decoder exists. □

Definition 7.4 (Recoverability preorder)**.** For a fixed task family $\mathcal{Q}$, write

$$R_1 \preceq_{\mathcal{Q}} R_2$$

when every task in $\mathcal{Q}$ that is recoverable from R_1 is also recoverable from R_2.

Proposition 7.5 (Recoverability is a preorder)**.** *The relation $\preceq_{\mathcal{Q}}$ is reflexive and transitive.*

Proof. Every representation supports every task it supports, so the relation is reflexive. If every task supported by R_1 is supported by R_2, and every task supported by R_2 is supported by R_3, then every task supported by R_1 is supported by R_3. Thus the relation is transitive. □

Define $R_1 \sim_{\mathcal{Q}} R_2$ when both $R_1 \preceq_{\mathcal{Q}} R_2$ and $R_2 \preceq_{\mathcal{Q}} R_1$.

Corollary 7.6 (Quotient poset)**.** *The recoverability preorder descends to a partial order on the equivalence classes under $\sim_{\mathcal{Q}}$.*

Proof. Reflexivity and transitivity pass to equivalence classes. Mutual recoverability has been quotiented out, so antisymmetry holds by construction. □

7.1 Pure retention operators

The simplest compression removes nodes without rewriting the retained material. This baseline is useful because it separates information deletion from more complicated summarization or reinterpretation.

Definition 7.7 (Retention projection)**.** Fix a finite node universe V. For a mask $K \subseteq V$, define

$$P_K(S) = S \cap K, \qquad S \subseteq V.$$

Theorem 7.8 (Retention projection algebra)**.** *For masks $K, L \subseteq V$,*

$$P_K \circ P_L = P_{K \cap L} = P_L \circ P_K.$$

Consequently every retention projection is idempotent, and the masks form a Boolean lattice with meet $K \cap L$ and join $K \cup L$.

Proof. For every $S \subseteq V$,

$$P_K(P_L(S)) = (S \cap L) \cap K = S \cap (K \cap L).$$

Intersection is associative and commutative, which gives the composition identity and commutativity. Taking $L = K$ gives idempotence. The power set of V with intersection, union, complement, bottom $\varnothing$, and top V is a Boolean lattice. □

Corollary 7.9 (Task monotonicity under retention)**.** *If $K \subseteq L$, every task whose required nodes and metadata are retained by P_K is also supported by P_L.*

Proof. Every item retained by K is retained by L. Hence every task requirement contained in K is contained in L. □

Pure retention operators commute. Noncommutativity in real proof workflows therefore requires an additional operation such as state-dependent summarization, renaming, quotienting, or reinterpretation. This distinction prevents simple deletion from being credited with structure it does not possess.

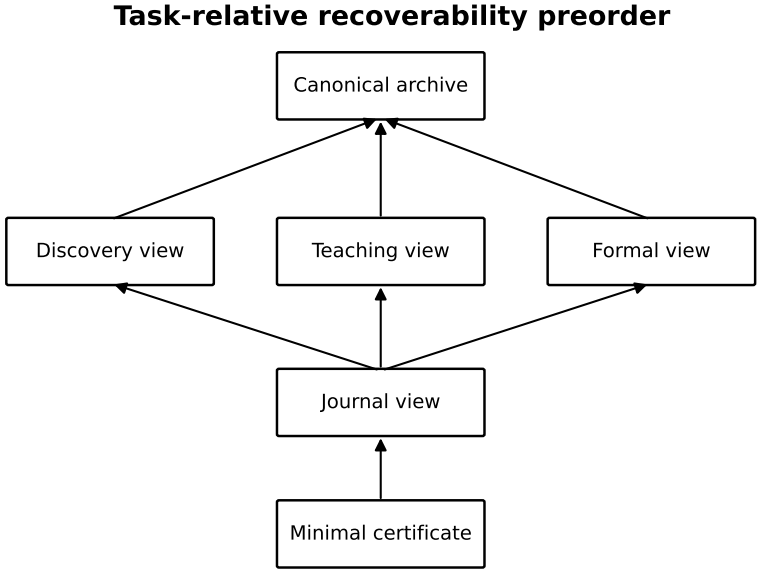


Figure 5: A task-relative recoverability preorder. Teaching, discovery, and formal views may be incomparable.

Theorem 7.10 (Task-family dependence of minimal compression)**.** *There need not be one proper compression that is simultaneously minimal and sufficient for every task family on a proof field.*

Proof. Consider a proof archive containing a certificate and two independent side records x and y. Let q_x ask for x and let q_y ask for y. For the singleton family $\{q_x\}$, a minimal sufficient representation retains the certificate and x but may omit y. For $\{q_y\}$, a minimal sufficient representation may omit x instead. For $\{q_x, q_y\}$, every sufficient representation must distinguish both records. Hence the minimal sufficient representations differ with the task family. Here minimality is relative to deletion of independently variable records and its stated retention cost. Sufficiency for all possible archive queries instead requires an injective representation. A lossless recoding can preserve every query while using a different, possibly shorter, encoding. The obstruction is identification of distinguishable archives, not a change of notation or file length. □

Theorem 7.11 (All predicates distinguish exactly injective encodings)**.** *Let $C : A \to B$. Every predicate $q : A \to \mathrm{Prop}$ factors through C if and only if C is injective.*

Proof. If C is injective, define the decoder of q at b by the predicate $\exists a$ ($C(a) = b$ and $q(a)$). At $b = C(a_0)$, injectivity makes this equivalent to $q(a_0)$. Conversely, if $C(a) = C(a')$, use the predicate $q(z) \equiv (z = a)$. Its decoded values at the equal encodings must agree, giving $a' = a$. This is the general collision criterion specialized to all predicates. □

The formal result `ProofPath.all_predicates_sufficient_iff_injective` records the corrected distinction between information loss and lossless recoding. It makes no assertion about how cheaply a decoder can be computed.

8 Endpoint, audit, and discovery equivalence

The phrase "the same proof" can refer to several distinct equivalence relations.

Definition 8.1 (Endpoint equivalence)**.** Two representations are endpoint equivalent when they certify the same declared premises and targets.

Definition 8.2 (Audit equivalence)**.** Two representations are audit equivalent when they preserve equivalent dependency, evidence, provenance, and scope information for checking the declared targets.

Definition 8.3 (Discovery equivalence)**.** Two representations are discovery equivalent relative to a declared bridge model when they preserve the same examples, alternatives, boundary cases, candidate routes, and future reach, up to the chosen graph equivalence.

When audit and discovery task families are required to include the endpoint tasks, either equivalence implies endpoint equivalence. Audit and discovery equivalence need not imply one another: a representation may preserve provenance without alternatives, or alternatives without provenance. Their joint task family is stronger than either. Any further hierarchy requires an explicit inclusion of task families. Two perfectly valid certificates can therefore expose different mathematical neighborhoods without differing in validity.

Remark 8.4. These equivalences are task-relative. Discovery equivalence requires a declared bridge model. It is not an assertion that every future use of a proof can be predicted.

9 Vector-valued loss

Let a node v carry nonnegative coordinates

$$\mathbf{v}(v) = (V(v), D(v), M(v), F(v), A(v)),$$

representing verification, discovery, meaning, future bridge, and auditability values. These coordinates are declared model inputs. They may be exact, estimated, or ordinal after a scale has been chosen.

Definition 9.1 (Additive loss vector)**.** For a retained node set $K \subseteq V$, define

$$\mathrm{Loss}(\mathcal{P}, K) = \sum_{v \notin K} \mathbf{v}(v).$$

When normalization is useful, divide each coordinate by the corresponding total coordinate of the full field.

Proposition 9.2 (Zero verification loss can coexist with positive discovery loss)**.** *There exist proof fields and certificates K for which the verification coordinate of* $\mathrm{Loss}(\mathcal{P}, K)$ *is zero while the discovery and bridge coordinates are positive.*

Proof. Take an AND-DAG certificate from A to B and add an optional example node E that is not an ancestor of B. Assign positive discovery and bridge values to E and zero verification value. The unique core omits E while preserving the entire verification ancestry. The verification loss is zero and the other declared losses are positive. □

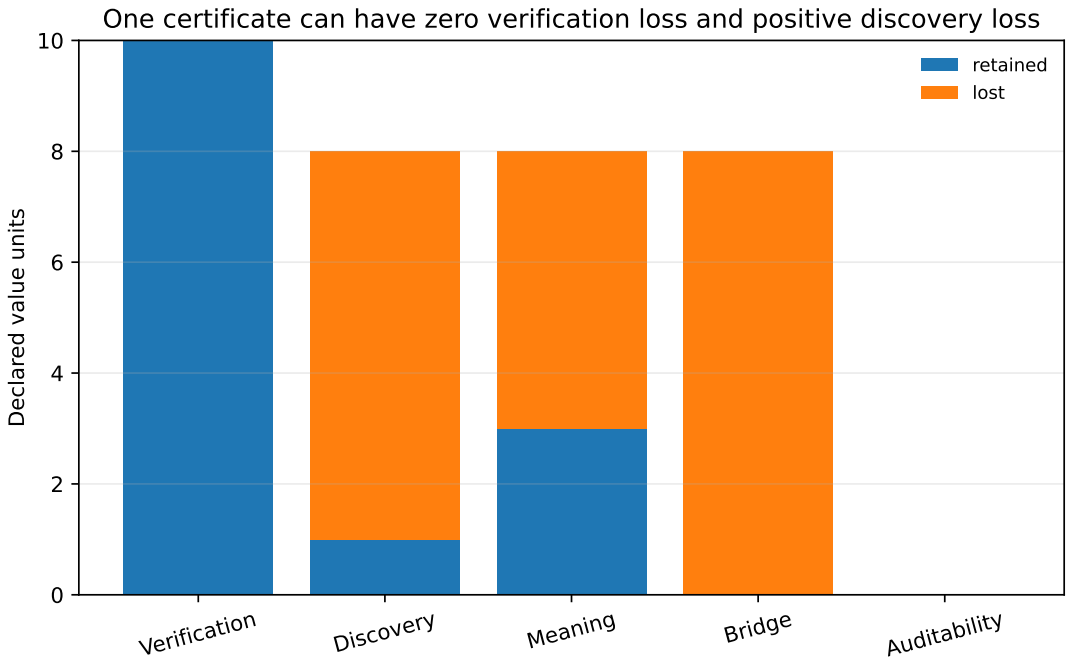


Figure 6: One finite example in which certificate extraction preserves verification and discards discovery value.

No claim is made that all value coordinates are additive. Additivity is a transparent baseline. Interactions can be represented by hyperedge values, submodular utilities, or task-specific decoders when the application requires them.

10 Compression debt

Compression may postpone work rather than eliminate it. A later task may require reconstruction of material that was omitted from the current representation.

Definition 10.1 (Future task model)**.** Let $\mathcal{T}$ be a finite task set with probabilities $p_t \geq 0$ satisfying $\sum_t p_t = 1$. Let $R_t \subseteq V$ be the node set required by task t, and let $c(v) \geq 0$ be the recovery cost of node v when it is absent.

Definition 10.2 (Expected compression debt)**.** The expected compression debt of retained set K is

$$\mathrm{Debt}(K) = \sum_{t \in \mathcal{T}} p_t \sum_{v \in R_t \setminus K} c(v).$$

Proposition 10.3 (Debt is monotone under retention)**.** *If $K_1 \subseteq K_2$, then*

$$\mathrm{Debt}(K_2) \leq \mathrm{Debt}(K_1).$$

Proof. For every task t, the missing set $R_t \setminus K_2$ is contained in $R_t \setminus K_1$. Recovery costs are nonnegative. The taskwise cost cannot increase, and neither can its probability-weighted sum. □

Corollary 10.4 (Irreversible compression accumulates debt)**.** *For a descending sequence of retained sets*

$$K_0 \supseteq K_1 \supseteq \cdots \supseteq K_r,$$

the expected debt is nondecreasing.

A recovery archive can change the cost function. If an omitted node has a stable provenance pointer and a reproducible generator, its recovery cost may be small. If it survives only in memory, its recovery cost may be effectively infinite.

11 A proof ecology rather than one ideal form

The same theorem can support several legitimate representations:

$$\mathcal{R}(B) = \{R_{\mathrm{cert}}, R_{\mathrm{journal}}, R_{\mathrm{teach}}, R_{\mathrm{formal}}, R_{\mathrm{discovery}}, R_{\mathrm{archive}}\}.$$

The certificate is optimized for checking. The journal form balances rigor and readable length. The teaching form expands conceptual bottlenecks. The formal form exposes primitive dependencies. The discovery form retains alternatives and failed routes. The archive preserves reconstruction information.

Definition 11.1 (Representation portfolio)**.** A representation portfolio is a family of proof representations together with stable claim identifiers, translation maps, and a shared status ledger.

Proposition 11.2 (Portfolio robustness)**.** *If several representations lose different information, then their union retains at least as much recoverable information as any one representation alone.*

Proof. Every item recoverable from one representation is recoverable from the union containing it. Additional representations may restore information absent from the first. Recoverable information is therefore monotone under enlargement of the portfolio. □

Redundancy is useful when it is heterogeneous. Storing six copies of the same compressed certificate adds little. Storing a certificate, an expanded explanation, a formal dependency graph, and a reproducible notebook may preserve genuinely different values.

12 Proof compilers and shared ledgers

A proof compiler generates a goal-specific representation from one canonical proof field.

Definition 12.1 (Proof compiler)**.** Let g be a presentation goal and let $\mathcal{P}$ be a versioned proof field. A proof compiler is a map

$$\Gamma_g : \mathcal{P} \longrightarrow R_g$$

that selects, orders, and renders the material required by g while preserving links to canonical claim and justification identifiers.

Typical goals include certificate, journal, teaching, discovery, formal verification, and archive. The compiler does not invent logical status. It reads status from the canonical ledger.

Proposition 12.2 (Shared-ledger consistency)**.** *Suppose every compiled representation is generated from one canonical ledger and every rendered claim retains its ledger identifier. Then a status correction can be propagated consistently to every compiled representation containing that claim by updating the ledger and recompiling.*

Proof. The stable identifier determines every occurrence of the claim across the compiled outputs. Updating the canonical record changes the source value associated with that identifier. Recompilation reads the updated record in every view. A stale status can persist only in an output detached from the canonical ledger or not recompiled. □

This proposition is simple, but it is operationally important. A monograph, a formal library, and a teaching site should not maintain independent truth labels for the same claim.

13 Latent bridge graphs

A proof field records certified dependencies. A latent bridge graph records possible future transfers. The two graphs have different logical roles.

Definition 13.1 (Latent bridge graph)**.** A latent bridge graph is a finite directed graph

$$\mathcal{B} = (W, E_B)$$

whose nodes include retained proof objects, omitted proof objects, and anticipated future targets. An edge $x \to y$ means that x has been declared a possible transfer route toward y. The edge may be certified, conjectural, or exploratory, but its status must be recorded.

Let $S \subseteq W$ be the retained starting set. Let $F \subseteq W$ be a future target set with weights $w_f \geq 0$.

Definition 13.2 (Future reach utility)**.** The weighted future reach of S is

$$U(S) = \sum_{f \in F} w_f \, \mathbf{1}\{f \in \mathrm{Reach}_{\mathcal{B}}(S)\}.$$

This quantity measures declared reach, not truth. A conjectural edge can increase modeled reach without certifying the target. The ledger must keep those statuses separate.

13.1 Bridge criticality and dominators

Definition 13.3 (Bridge-critical node)**.** A node v is bridge critical for target f relative to starts S when f is reachable from S in $\mathcal{B}$ and becomes unreachable after v is removed.

In graph language, v dominates f relative to S when every directed path from S to f passes through v.

Theorem 13.4 (Criticality equals domination)**.** *Assume f is reachable from S. A node $v \notin S$ is bridge critical for f if and only if v dominates f relative to S.*

Proof. If v dominates f, every path from S to f uses v. Removing v removes every such path, so f becomes unreachable. Conversely, if removing v makes f unreachable, then there could not have been a path from S to f avoiding v. Therefore every path used v, which is exactly domination. □

Corollary 13.5 (Weighted criticality)**.** *The utility lost by removing v is the total weight of future targets dominated by v:*

$$U(S) - U_{\mathcal{B}\setminus\{v\}}(S) = \sum_{f \in F:\, v \text{ dominates } f} w_f.$$

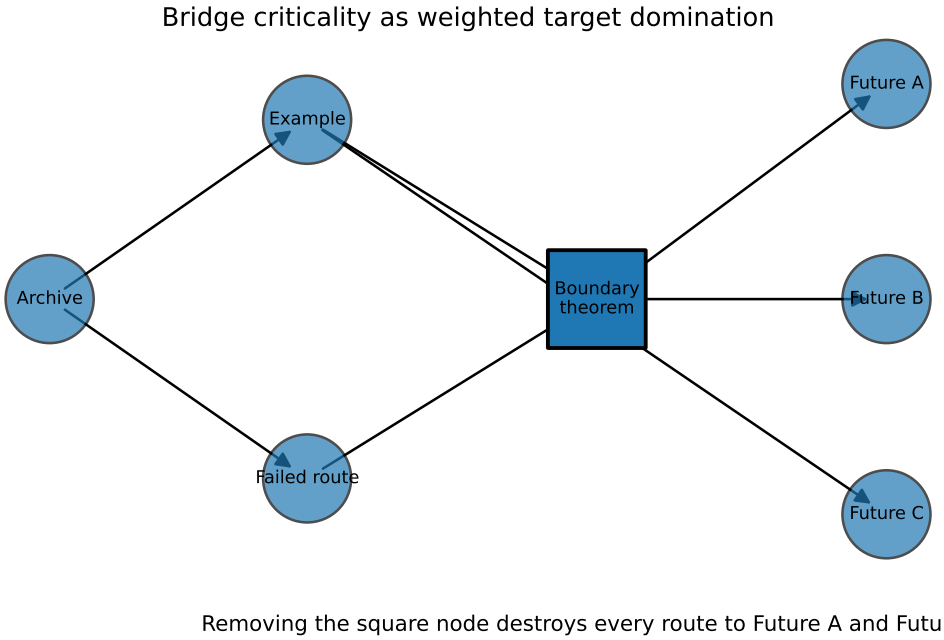


Figure 7: A bridge-critical node dominates two future targets.

A node may have low verification value for the current theorem and high domination value for future targets. This is the precise form of the claim that a forgotten step can remain mathematically important after it has lost its role in the original certificate.

14 Retention under a budget

An archive cannot retain every object at full resolution. The first mathematically controlled retention problem is to choose a finite supplement that preserves as much future reach as possible.

Let $C \subseteq W$ be the candidate nodes that may be retained and let M be a mandatory base set. For $S \subseteq C$, define

$$U_M(S) = U(M \cup S).$$

Theorem 14.1 (Bridge coverage is monotone and submodular)**.** *The future reach utility U_M is a nonnegative monotone submodular set function on C.*

Proof. For each candidate $x \in C$, let $R_x \subseteq F$ be the set of future targets reachable from $M \cup \{x\}$. Let R_M be the targets already reachable from M. Reachability from a union of starting nodes is the union of their individual reach sets, so

$$U_M(S) = \sum_{f \in R_M \cup \bigcup_{x \in S} R_x} w_f.$$

Adding a candidate cannot remove a reachable target, so U_M is monotone.

Now let $S \subseteq T \subseteq C$ and let $x \in C \setminus T$. The marginal gain of adding x is the total weight of targets in R_x that are not already covered. Since T covers at least every target covered by S, the set newly covered from T is contained in the set newly covered from S. Therefore

$$U_M(S \cup \{x\}) - U_M(S) \geq U_M(T \cup \{x\}) - U_M(T),$$

which is submodularity. □

Theorem 14.2 (Greedy cardinality guarantee)**.** *Let S_k be the set obtained by starting from the empty supplement and repeatedly adding the candidate with largest marginal future reach until k candidates have been selected. Let O_k be an optimal supplement of size at most k. Then*

$$U_M(S_k) - U_M(\varnothing) \geq \left(1 - \left(1 - \frac{1}{k}\right)^k\right) (U_M(O_k) - U_M(\varnothing)),$$

and therefore the greedy gain is at least $(1 - 1/e)$ times the optimal gain.

Proof. Write $G_i = U_M(S_i) - U_M(\varnothing)$ and $G^* = U_M(O_k) - U_M(\varnothing)$. By monotonicity and submodularity, the total marginal contribution of the at most k elements of O_k beyond S_i is at least $G^* - G_i$. Hence at least one candidate has marginal gain at least $(G^* - G_i)/k$. Greedy chooses a candidate with at least that gain, so

$$G^* - G_{i+1} \leq \left(1 - \frac{1}{k}\right) (G^* - G_i).$$

Iterating from $G_0 = 0$ gives

$$G^* - G_k \leq \left(1 - \frac{1}{k}\right)^k G^*.$$

Rearranging gives the first bound. The inequality $(1 - 1/k)^k \leq e^{-1}$ gives the second. □

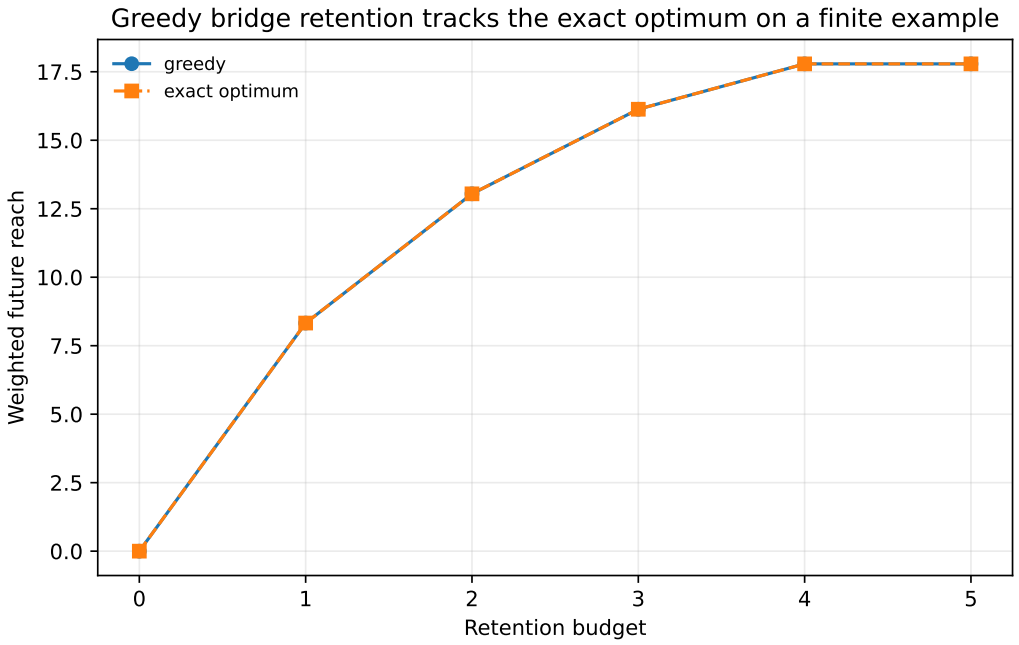


Figure 8: Greedy and exact bridge retention on one reproducible finite example. The theorem gives a worst-case guarantee. The displayed instance performs better.

Warning 14.3 (Arbitrary costs). The simple marginal-gain-per-cost rule included in the software is an interpretable heuristic for arbitrary real costs. The cardinality guarantee above does not automatically apply to that heuristic. Stronger knapsack guarantees require additional algorithmic machinery.

15 Versioned proof objects

A proof archive changes. Claims are strengthened, corrected, refuted, or split. A reproducible theory therefore needs stable identifiers and explicit version relations.

Definition 15.1 (Versioned proof object)**.** A versioned proof object is a sequence

$$\mathcal{P}_0 \longrightarrow \mathcal{P}_1 \longrightarrow \cdots$$

whose nodes and justifications carry stable identifiers. Each transition records additions, removals, altered scopes, evidence changes, and a digest of its parent version.

Definition 15.2 (Append-only certified update)**.** An update $\mathcal{P}_i \to \mathcal{P}_{i+1}$ is append-only certified when every node and justification of $\mathcal{P}_i$ remains unchanged, and $\mathcal{P}_{i+1}$ only adds nodes, evidence, or certified justifications.

Theorem 15.3 (Derivability is monotone under append-only updates)**.** *Every target derivable in $\mathcal{P}_i$ remains derivable in $\mathcal{P}_{i+1}$ under an append-only certified update.*

Proof. A certificate in $\mathcal{P}_i$ uses only nodes and certified justifications that remain unchanged in $\mathcal{P}_{i+1}$. The same selected subfield therefore satisfies the same derivability closure in the later version. □

Retractions and semantic changes are not append-only. They require invalidation analysis.

Definition 15.4 (Syntactic affected set)**.** For a changed node set $X \subseteq V$, define

$$\mathrm{Aff}(X) = X \cup \mathrm{Desc}(X),$$

where descendants are computed in the declared dependency graph.

Theorem 15.5 (Minimal closed syntactic review set)**.** *The set* $\mathrm{Aff}(X)$ *is the unique smallest node set* Y *such that* $X \subseteq Y$ *and* Y *is closed under outgoing dependency edges: whenever* $u \in Y$ *and* $u \to v$ *is a declared dependency edge, then* $v \in Y$*.*

Proof. By construction, every node reachable from X lies in $\mathrm{Aff}(X)$, so the set contains X and is closed under outgoing edges. Let Y be any other set with those two properties. Every directed path beginning in X remains inside Y by induction on path length. Hence every descendant of X lies in Y, and $\mathrm{Aff}(X) \subseteq Y$. □

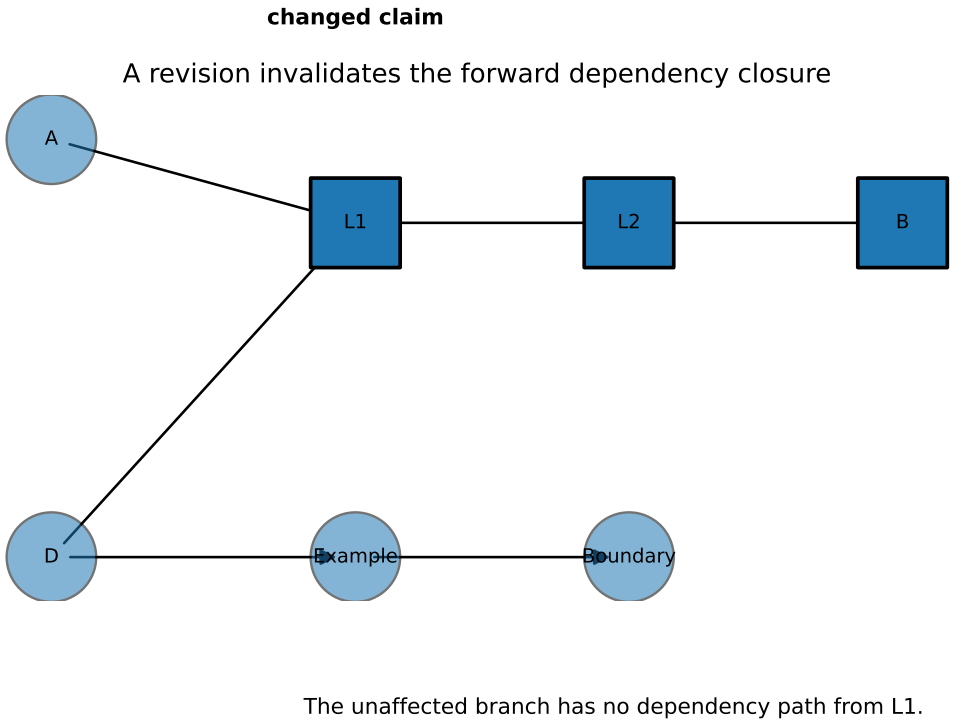


Figure 9: A changed claim and its forward dependency closure.

The theorem is intentionally syntactic. A descendant can survive after rechecking. The affected set says where review is required, not where falsehood is guaranteed.

16 Evidence ledgers and provenance

A claim ledger should answer five questions without reconstructing the entire research history.

1. What is the exact claim identifier?

2. What is its current status and scope?
3. Which justifications depend on it?
4. Which evidence records support or challenge it?
5. Which versions and compiled documents contain it?

Definition 16.1 (Evidence ledger)**.** An evidence ledger is an appendable collection of typed, scoped records linked to claim and justification identifiers. A record may certify, support, challenge, interpret, or refute an object. Its role is explicit rather than inferred from prose placement.

Definition 16.2 (Provenance digest)**.** A provenance digest is a cryptographic hash of a canonical serialization of the claim identifier, content, scope, evidence references, parent version, and relevant build inputs.

A digest does not prove a theorem. It detects unrecorded changes and gives independently generated representations a common reference point.

Proposition 16.3 (Digest stability)**.** *A deterministic canonical serialization gives the same provenance digest for identical records and, except for hash collisions, a different digest after any serialized field changes.*

Proof. Deterministic serialization produces the same byte sequence for identical records, so the hash is identical. Any change that alters the canonical byte sequence changes the hash input. A collision-resistant hash is designed to make equal hashes for distinct byte sequences computationally negligible, though not logically impossible. □

17 Correlated review

Several reviews are not equivalent to several independent reviews. Reviewers may share training, notation, examples, software, or an unnoticed premise.

Let $X_i = \theta + \varepsilon_i$ be reviewer estimates of a quantity θ, with common variance σ^2 and common pairwise correlation ρ. For $n > 1$, assume $-1/(n-1) < \rho \leq 1$, so the mean has positive variance. The mean has variance

$$\operatorname{Var}(\bar{X}) = \sigma^2 \left(\frac{1-\rho}{n} + \rho \right).$$

Theorem 17.1 (Effective reviewer count)**.** *Under the equicorrelated model, the reviewer mean has the same variance as the mean of*

$$\boxed{n_{\text{eff}} = \frac{n}{1 + (n-1)\rho}}$$

independent reviewers of variance σ^2.

Proof. The mean of m independent reviewers of variance σ^2 has variance σ^2/m. Equating this with the correlated mean variance gives

$$\frac{\sigma^2}{m} = \sigma^2 \left(\frac{1-\rho}{n} + \rho \right) = \sigma^2 \frac{1 + (n-1)\rho}{n}.$$

Solving for m gives the formula. □

For fixed positive ρ, n_{eff} approaches $1/\rho$ as n grows. Headcount continues to help, but shared dependence creates a saturation floor.

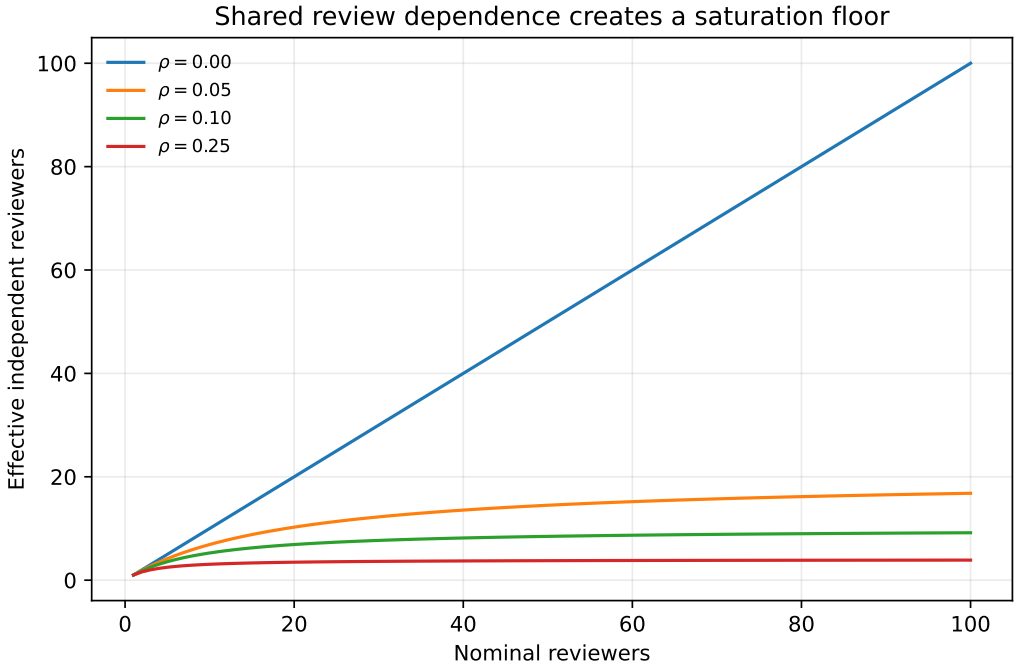


Figure 10: Effective independent reviewer count under equicorrelated errors.

For a general positive definite covariance matrix Σ, consider unbiased linear aggregates $w^\top X$ with $\mathbf{1}^\top w = 1$.

Theorem 17.2 (Minimum-variance covariance-aware weights)**.** *The unique minimum-variance unbiased linear aggregate has weights*

$$w^* = \frac{\Sigma^{-1}\mathbf{1}}{\mathbf{1}^\top \Sigma^{-1}\mathbf{1}}.$$

Its variance is

$$\frac{1}{\mathbf{1}^\top \Sigma^{-1}\mathbf{1}}.$$

Proof. Minimize $w^\top \Sigma w$ subject to $\mathbf{1}^\top w = 1$. The Lagrangian is

$$L(w, \lambda) = w^\top \Sigma w - 2\lambda(\mathbf{1}^\top w - 1).$$

Stationarity gives $\Sigma w = \lambda \mathbf{1}$, hence $w = \lambda \Sigma^{-1}\mathbf{1}$. The constraint gives

$$\lambda = (\mathbf{1}^\top \Sigma^{-1}\mathbf{1})^{-1}.$$

Positive definiteness makes the objective strictly convex, so the solution is unique. Substitution gives the variance formula. □

Warning 17.3 (Bias is not covariance)*.* Covariance-aware weighting does not remove a common bias that is absent from the covariance model. If every reviewer shares bias b, the mean squared error contains the irreducible term b^2.

The practical conclusion is not that consensus is useless. It is that review portfolios should diversify dependence structure, not only increase nominal count.

18 Verification bandwidth and audit queues

A system can generate mathematical candidates faster than it can verify them. Let a_t be arrivals during audit cycle t, let s_t be service capacity, and let B_t be the unresolved backlog. The basic deterministic queue is

$$B_{t+1} = \max\{0, B_t + a_t - s_t\}.$$

Theorem 18.1 (Constant-rate audit regimes)**.** *Suppose $a_t \equiv \lambda$ and $s_t \equiv \mu$.*

(i) *If $\lambda < \mu$, every finite initial backlog reaches zero after finitely many cycles.*

(ii) *If $\lambda = \mu$, the backlog remains constant.*

(iii) *If $\lambda > \mu$, the backlog grows linearly with slope $\lambda - \mu$.*

Proof. Before the truncation at zero, each cycle adds $\lambda - \mu$. When this quantity is negative, the backlog decreases by a fixed amount until it reaches zero. When it is zero, the backlog is unchanged. When it is positive, the truncation is never active after a nonnegative start, and the backlog is $B_0 + t(\lambda - \mu)$. □

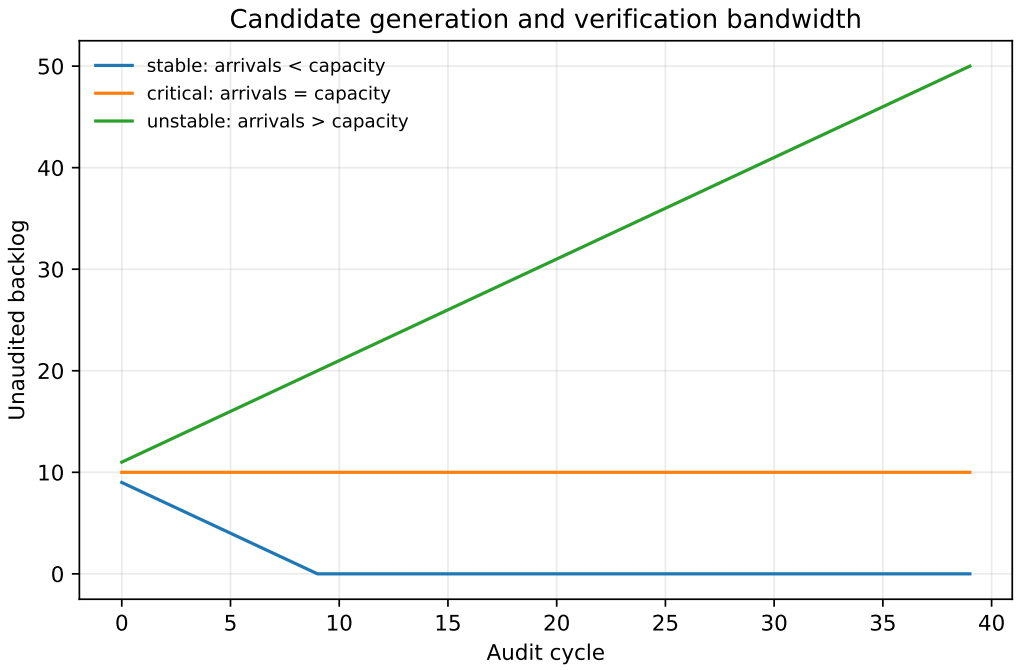


Figure 11: Stable, critical, and unstable audit regimes in the constant-rate model.

A simple audit priority score is

$$P(x) = \frac{p_{\mathrm{err}}(x)\, c_{\mathrm{consequence}}(x)\big(1 + r_{\mathrm{dependency}}(x)\big)}{c_{\mathrm{audit}}(x)}.$$

This is a decision model, not a truth measure. It makes the assumptions behind triage explicit.

19 Recursive proof cubes and expansion depth

A visible proof step is rarely atomic. A lemma opens into definitions and sublemmas. A computation opens into algorithms, parameters, and exact checks. A citation opens into another dependency graph. Proof Path Theory treats this as recursive refinement.

Definition 19.1 (Proof packet and refinement)**.** A proof packet is a node presented at one resolution. A refinement replaces that node by a finite subfield whose certified target reconstructs the original node on the declared scope.

Repeated refinement gives a nested sequence of representations

$$R_0 \prec R_1 \prec \cdots \prec R_d,$$

where deeper levels expose more internal structure. Let $r_d \geq 0$ be the unresolved residual at depth d and let $c_d \geq 0$ be the cumulative cost.

Definition 19.2 (Goal-relative depth objective)**.** For goal g with weights $\alpha_g, \beta_g \geq 0$, define

$$J_g(d) = \alpha_g r_d + \beta_g c_d.$$

An optimal expansion depth is any minimizer of J_g.

Theorem 19.3 (Expansion depth is goal dependent)**.** *The same proof object can have different optimal expansion depths for different goals.*

Proof. It is enough to construct a proof object with exactly two available depths. Choose $d_1 < d_2$ with $r_{d_2} < r_{d_1}$ and $c_{d_2} > c_{d_1}$. Goal g prefers d_2 over d_1 exactly when

$$\frac{\alpha_g}{\beta_g} > \frac{c_{d_2} - c_{d_1}}{r_{d_1} - r_{d_2}},$$

with the usual interpretation when $\beta_g = 0$. Choose one goal ratio below this threshold and another above it. The first prefers the shallower depth and the second prefers the deeper depth. □

A practical stopping rule selects the first depth satisfying either

$$r_d \leq \varepsilon$$

or

$$\frac{r_{d-1} - r_d}{c_d - c_{d-1}} < \tau.$$

The first condition stops when the residual is small enough. The second stops when the next refinement gives too little residual reduction per unit cost.

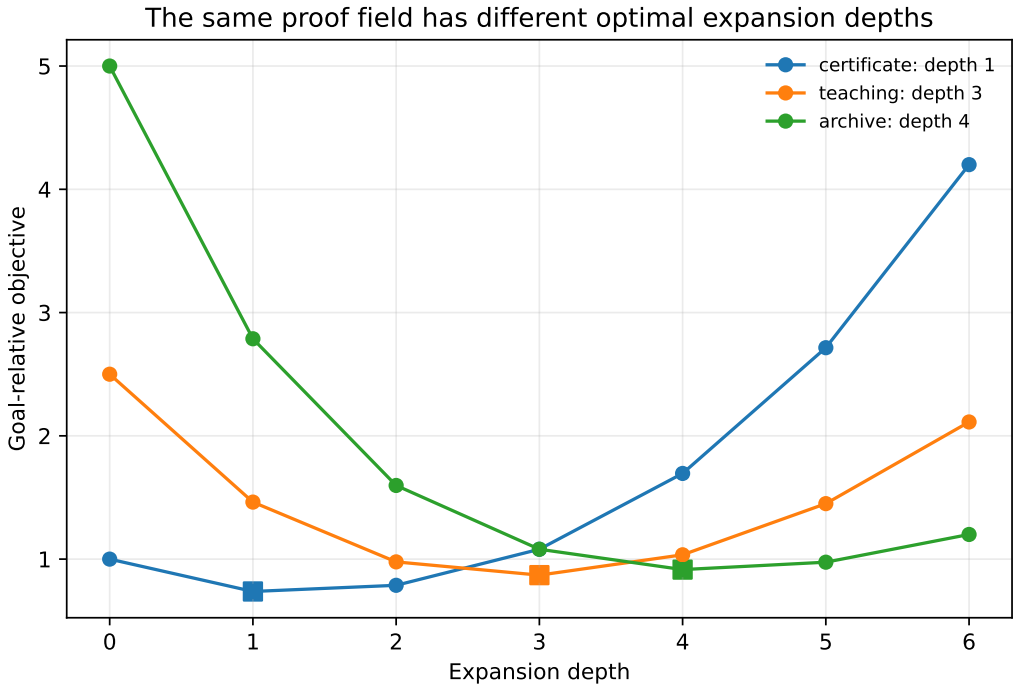


Figure 12: Certificate, teaching, and archive goals select different depths in one finite model.

Deeper is not automatically better. A formal checker may need primitive steps. A specialist paper may cite a standard theorem. A teaching account may expand only the conceptual bottleneck. An archive may retain a failed branch that neither certificate requires.

20 Worked finite examples

The accompanying repository contains three canonical JSON proof fields and several auxiliary models. The examples are small enough to inspect by hand and complete enough to exercise the formal distinctions.

20.1 A fixed AND-DAG

The field `examples/and_dag.json` has required premise A, a definition D, lemmas L_1 and L_2, and target B. It also contains a worked example, a failed route, and a boundary case. The unique verification core is

$$\{A, D, L_1, L_2, B\}.$$

The optional nodes have zero declared verification value for B and positive discovery, meaning, or bridge value. This is the executable instance behind Figure 6.

20.2 Two certified proofs

The field `examples/alternative_proofs.json` contains a short route and a richer explanatory route. The short route has storage cost 1, while the richer route has storage cost 3 and larger meaning and

discovery values. Exact certificate search chooses

$$A \longrightarrow L_{\text{short}} \longrightarrow B$$

when the objective is storage cost alone. A teaching compiler may rationally choose the other route because it uses a different objective.

20.3 Set Cover encoded as certificate selection

The field `examples/set_cover_certificate.json` contains four optional source lemmas and four obligations. Exact search returns the certificate using S_1 and S_3, with total cost 2. The particular optimum is not important. The example demonstrates how local alternative derivations create a global combinatorial selection problem.

20.4 Bridge retention

The bridge experiment uses ten candidate nodes and sixteen weighted future targets. For budgets zero through five, the script computes both the exact optimum and the greedy solution. The generated data are stored in `data/retention_comparison.csv`. On this instance the two curves coincide. The theorem guarantees only the worst-case $(1 - 1/e)$ factor.

20.5 Correlated review and audit capacity

For one hundred equicorrelated reviewers with $\rho = 0.1$,

$$n_{\text{eff}} = \frac{100}{1 + 99(0.1)} \approx 9.17.$$

The audit example begins with backlog 10. Arrivals 4 and capacity 5 clear the queue. Equal rates hold the backlog at 10. Arrivals 6 and capacity 5 increase it by one per cycle.

Table 1: Canonical finite checks in the version 1.0 release.

Object	Computed result	Mathematical role
AND-DAG	unique five-node core	Theorem 4.3
Alternative field	minimum cost 1	exact finite search
Set Cover field	minimum cost 2	Theorem 5.2 encoding
Bridge model	greedy and exact curves	Theorems 14.1–14.2
Review model	n_{eff} formula	Theorem 17.1
Audit model	three queue regimes	Theorem 18.1

21 Reference implementation

The repository implements the finite theory as the Python package `proof_path_theory`. The public modules are organized by mathematical responsibility.

Module	Responsibility
`model.py`	Typed nodes, justifications, evidence records, acyclicity, ancestry, and finite derivability.
`certificate.py`	Certificate validation, unique AND-DAG cores, exact minimum certificate search, and certificate cost.
`compression.py`	Task sufficiency, recoverability preorder, loss reports, compression debt, representation utilities, and Pareto fronts.
`bridges.py`	Reachability, domination-based criticality, exact retention, greedy retention, and finite submodularity checks.
`review.py`	Equicorrelated covariance, effective reviewer count, generalized least squares, and shared-bias mean squared error.
`audit.py`	Audit queue simulation, rate regimes, and explicit priority scores.
`refinement.py`	Goal-relative refinement decisions, optimal depth, and stopping rules.
`versioning.py`	Affected descendants, append-only checks, ledger snapshots, and provenance digests.
`ambient.py`	Finite declared view fibers, restriction checks, and arrow-cover gluing.
`realization.py`	Exact binary elimination, protected fibers, obstruction witnesses, and observation gain.
`views.py`	Local field models, whole-support gluing, declared countermodels, and intersection diagrams.
`io.py`	Stable JSON import and export for proof fields and certificates.
`cli.py`	Command-line validation, core extraction, minimum certificate search, and certificate audit.

21.1 Command-line interface

The main commands are:

```
proof-path validate examples/and_dag.json
proof-path core examples/and_dag.json --output core.json
proof-path minimum examples/alternative_proofs.json --output minimum.json
proof-path audit examples/and_dag.json core.json
```

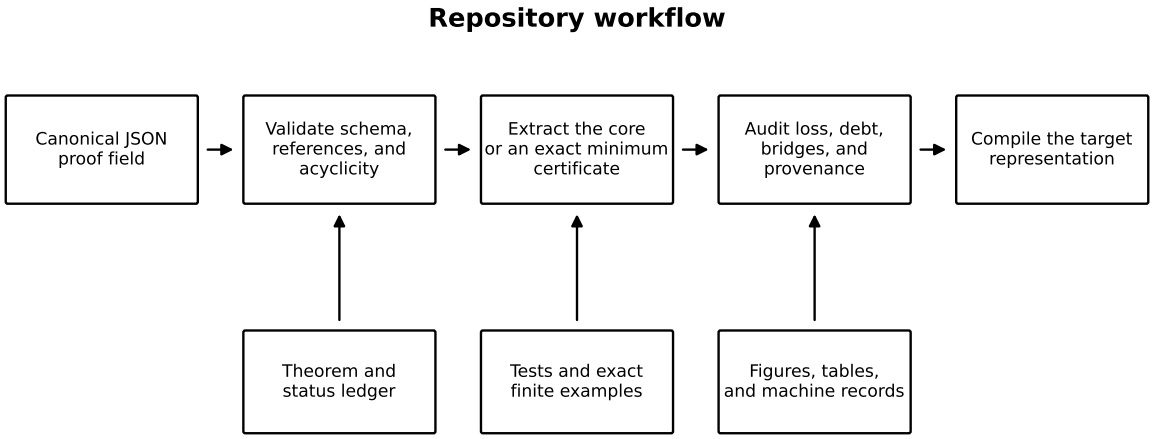


Figure 13: Repository workflow from a canonical JSON field to compiled representations and reproducible records.

21.2 JSON contract

A proof field file contains:

- a schema version and metadata;
- required premises and targets;
- typed nodes with value and cost vectors;
- typed evidence records with scope and certification flags;
- justifications with premise sets, conclusions, rules, evidence references, and certification flags.

The schema is intentionally plain. It can be produced by a proof assistant adapter, a research notebook, a theorem ledger, or a hand-written example.

21.3 Tests and build discipline

Version 1.2 expands the original seventy-four-test suite with bridge, schema-boundary, and input-validation regressions. The exact executed count is recorded in the release verification report. They cover graph validation, derivability, certificate extraction, alternative proofs, the Set Cover encoding, loss vectors, task sufficiency, bridge submodularity, retention, review covariance, audit queues, refinement, version propagation, JSON round trips, and command-line smoke tests.

A separate deterministic cross-check compares the exact certificate solver with brute-force enumeration over eighteen cost profiles, tests the greedy cardinality guarantee on one hundred generated coverage instances, verifies 1,024 retention-algebra identities, and compares the effective-reviewer formula with direct covariance calculations in 114 parameter settings.

The tests and cross-checks verify the implementation of finite definitions and examples. They do not replace the proofs in this paper. Conversely, the proofs do not guarantee that a software implementation

is free of defects. The release keeps both forms of verification.

22 The ambient proof field and the meaning of failure

A finished proof is rarely the only route the researcher could have explored. An archive often contains sketches that were never completed, arguments that failed because of a specific hypothesis, conjectured bridges that remain untested, and genuinely refuted local constructions. It is tempting to call this whole archive a proof, but doing so would erase the distinction between a certified result and a record of exploration. The *ambient proof field* provides a more precise language: it is the declared field of reachable or recorded mathematical states and typed candidate transitions in which certified proof paths form a distinguished subfield.

This section sharpens an idea already implicit in the finite hypergraph model. The existing `ProofField` permits `failed_route`, `boundary_case`, and `counterexample` nodes. Their presence is useful, but none is a logically justified global nonexistence claim by itself. A failed search is not necessarily a proof of impossibility. A counterexample to one proposed lemma is not necessarily a counterexample to the goal. An exact obstruction must have a declared scope and a checkable witness.

Definition 22.1 (Scoped branch assessment)**.** Let $\mathcal{P}$ be a finite typed proof field with declared premises A and target t. An exploratory branch is a finite sequence of candidate justifications whose certification and outstanding obligations are explicitly recorded. Its assessment is one of the following, relative to stated premises and rules:

(i) *Open*: obligations remain unresolved.

(ii) *Abandoned*: exploration stopped without a certificate of impossibility.

(iii) *Locally refuted*: a checked counterexample or obstruction excludes a stated intermediate step under specified hypotheses.

(iv) *Certified*: the selected justifications meet the declared proof certificate conditions.

An assessment is a record of the branch, not a truth value assigned to every possible proof of t.

Proposition 22.2 (Local failure does not imply global failure)**.** *A finite proof field may contain a locally refuted branch to target t while also containing a certified derivation of t from its declared premises.*

Proof. Take a premise A and two alternative candidate derivations of t. Let the first use a proposed intermediate lemma L that has a checked counterexample, so the corresponding justification is not certified. Let the second use a distinct certified justification from A to t. The certified closure reaches t by the second route. The failed first route remains a valid record of unsuccessful exploration, but cannot negate the existence of the second certificate. The same example can be represented by the alternative justification hypergraph developed earlier. □

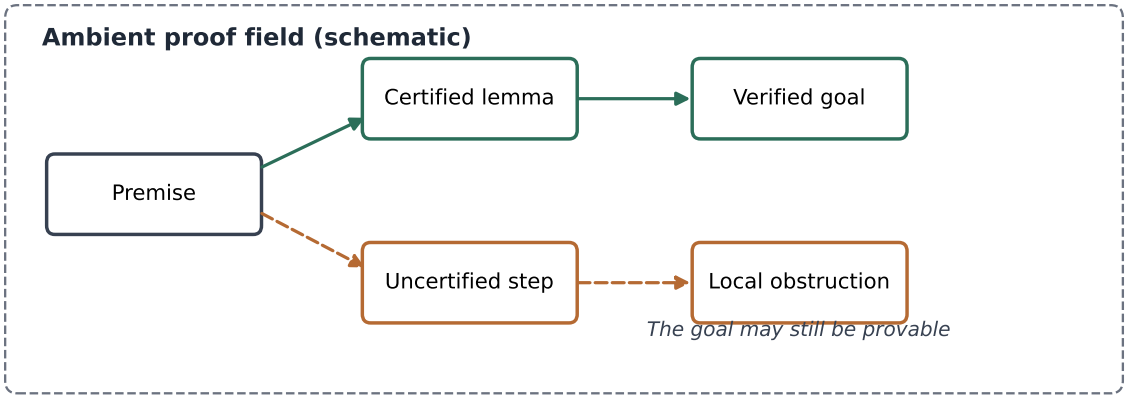


Figure 14: A certified derivation and a locally blocked candidate route coexist in the same declared proof field. The blocked branch is not a certificate that the goal itself is impossible. Generated by the supplied Python script.

The logical lesson is substantive for machine reasoning. A search timeout, budget exhaustion, invalid intermediate conjecture, or failure to find a proof all have different meanings. Only a certificate whose conclusion explicitly concerns nonexistence under a declared model and scope licenses a nonexistence claim. This is especially important when a failed branch is used to prune an automated search. Pruning the refuted step may be safe, while pruning its entire goal may discard a genuine proof.

Proposition 22.3 (Scoped refutation survives unchanged context)**.** *Suppose a witness w is verified by a sound checker to refute the predicate L under assumptions A. In any later archive revision that retains the same interpretation of L, the same assumptions, and the checker's verification rules, w remains a refutation of L. No conclusion follows about different predicates, altered assumptions, or alternative branches to a different goal.*

Proof. The verification of w depends only on the declared predicate, assumptions, and checking rules. An append-only collection of unrelated exploratory records does not alter those inputs. Therefore the same verification remains valid. When any of those inputs changes, the earlier check no longer certifies the revised claim without an explicit comparison theorem. □

This is a qualified stability principle, not a general theorem that proof search is decidable. The append-only derivability result of Section 15 concerns certified edges. The present proposition concerns the narrower persistence of a checked counterexample under a fixed interpretation. Both retain scope as part of the mathematical data.

23 Komorebi Space, realization fibers, and informative questions

Komorebi Space supplies a natural setting for studying the proof field's coherent realizations. Here the connection is *relative* to an explicitly chosen system of views. Nothing in a finite proof hypergraph canonically supplies one universal category of all proofs, and a formal proof term is not automatically equivalent to an intended semantic object. The view system must be stated, together with its restrictions and the observations that are required to survive.

Definition 23.1 (Coherent views of a proof field)**.** Choose a finite collection of views V, fibers F_v of local proof states or annotations, and for each declared directed edge $e : v \to w$ a restriction map $e^* : F_w \to F_v$. The *strict realization set* is

$$\Gamma(F) = \{(x_v) \in \prod_{v \in V} F_v : e^*(x_w) = x_v \text{ for every declared edge } e\}.$$

The *extension fiber* of prescribed information s on $U \subseteq V$ consists of those global coherent families whose restrictions equal s. If the edges form a category, the usual functorial composition equations are additional explicit obligations.

The finite implementation `ambient.py` realizes this edge-defined model exactly. The gluing condition should not be weakened to covering the objects alone: a compatibility equation is attached to an edge and must appear in at least one patch.

Theorem 23.2 (Finite arrow-cover gluing)**.** *Suppose patches $U_i \subseteq V$ cover every view and every declared edge has both endpoints in some U_i. Let s_i be coherent assignments on U_i that agree on all overlaps. There is a unique global coherent assignment $s \in \Gamma(F)$ restricting to every s_i.*

Proof. For each view v, choose any patch containing it and define $s(v)$ to be the value supplied by that patch. Agreement on overlaps makes this independent of the choice. For a declared edge $e : v \to w$, choose a patch containing both endpoints. The local assignment on that patch obeys $e^*s(w) = s(v)$, so the global assignment is coherent. It is the only possible glue because every coordinate was prescribed by at least one patch. □

Example 23.3 (Object coverage cannot replace arrow coverage)**.** Let $V = \{v, w\}$ with an edge $v \to w$, both fibers $\{0, 1\}$, and identity restriction. Singleton patches prescribe $s(v) = 0$ and $s(w) = 1$. These assignments cover all views and agree on their empty overlap. There is no global coherent assignment with those coordinates. The missing ingredient is the edge condition, not the availability of the two individual values.

The theorem is a finite instance of the strict gluing construction in *Komorebi Space* [1]. It does not claim a general stack theorem for nonabelian proof objects. Gluing up to isomorphism requires the identifications and their coherence witnesses, which are retained in the corresponding groupoid setting.

23.1 Exact finite realization and obstruction certificates

A second connection is quantitative. Work over $\mathbb{F}_2$ with $E = \mathbb{F}_2^n$. Let $A : E \to \mathbb{F}_2^m$ impose coherence, $P : E \to \mathbb{F}_2^p$ retain prescribed information, and $x \in E$ be a raw candidate. Define

$$\mathfrak{K}_P(x) = \{y \in E : Ay = 0,\ Py = Px\}, \qquad M = \begin{bmatrix} A \\ P \end{bmatrix}, \quad b = \begin{bmatrix} 0 \\ Px \end{bmatrix}.$$

This is a finite linear realization fiber. The statement $My = b$ does not automatically produce a solution; compatibility of the prescribed information must first be established.

Theorem 23.4 (Finite realization alternative and ambiguity)**.** *Exactly one of the following holds:*

(i) *$My = b$ has a solution y_0. Its complete solution set is $y_0 + H$, where $H = \ker A \cap \ker P$, and*

$$|\mathfrak{K}_P(x)| = 2^{n - \operatorname{rank}(M)}.$$

(ii) *A row vector λ exists with $\lambda M = 0$ and $\lambda b = 1$. It is a directly checkable certificate that $\mathfrak{K}_P(x) = \varnothing$.*

Proof. If $My_0 = b$, any other solution differs from y_0 by an element of $\ker M = H$, and every such difference gives another solution. Rank nullity gives $\dim H = n - \operatorname{rank}(M)$. If b is not in the column space of M, linear algebra supplies a linear functional that vanishes on that column space and takes value 1 on b. Expressing it as a row vector gives λ. Conversely such a witness contradicts any proposed equation $My = b$, since it would imply $0 = \lambda My = \lambda b = 1$. These cases are mutually exclusive and exhaustive over a field. □

This negative certificate is a concrete instance of the protected obstruction principle from Komorebi Space. Its scope is exactly the finite linear system $My = b$. It is not a certificate that some broad mathematical conjecture is unprovable.

23.2 The information gained by a new observation

Suppose the realization fiber is nonempty and a new linear observation $Q : E \to \mathbb{F}_2^q$ becomes available. The right question is not how many bits appear in the output of Q but how many remaining degrees of freedom in the fiber it distinguishes.

Theorem 23.5 (Exact observation gain)**.** *Give the nonempty finite affine fiber $y_0 + H$ the uniform distribution. Then the Shannon entropy of the observed value Qy is*

$$H(Qy) = \operatorname{rank}(Q|_H) \quad \textit{bits.}$$

For every observation value that actually occurs, the corresponding conditional fiber contains

$$2^{\dim H - \operatorname{rank}(Q|_H)}$$

realizations. Thus the uncertainty reduction is exactly $\operatorname{rank}(Q|_H)$ *bits.*

Proof. The image $Q(H)$ is a vector subspace of dimension $r = \operatorname{rank}(Q|_H)$, so the observed values form the affine set $Qy_0 + Q(H)$ of size 2^r. Every such value has a preimage that is a coset of $H \cap \ker Q$, whose dimension is $\dim H - r$ by rank nullity. Equal fiber cardinalities make the induced output distribution uniform on 2^r values. Its entropy is $\log_2(2^r) = r$, and each conditional fiber has cardinality $2^{\dim H - r}$. □

Example 23.6 (One informative question)**.** Let $y = (a, b, c) \in \mathbb{F}_2^3$ with $a + b + c = 0$ and protected value $a = 0$. The realization fiber is $\{(0, 0, 0), (0, 1, 1)\}$. It has one bit of ambiguity. Observing b resolves that bit exactly. Observing a supplies no new information, despite being a perfectly valid measurement. This difference is measured by the rank of the observation restricted to H, rather than by the output width of the observation.

The new module `realization.py` computes these quantities through exact binary row reduction, and returns an explicit linear witness when the protected realization is impossible. The program does not enumerate all solutions, although the tests compare its results with exhaustive enumeration on small cases. The entropy statement assumes uniform sampling of the compatible realizations. It does not assert a universal prior over mathematical discoveries, or that the most informative observation is automatically the most valuable research question once costs and task priorities are included.

24 A checked realization bridge for proof fields

The ambient interpretation can be made into a mathematical construction rather than left as an analogy. The important step is to specify what a local realization means. A node label may describe an arbitrary mathematical sentence, but a Boolean value attached to that node does not automatically evaluate that sentence. In this section it records membership in a set closed under the declared certified rules. This restricted interpretation has an exact soundness and completeness theorem. An interpretation in actual mathematical propositions is a separate contract.

24.1 The declared rule system and its models

Fix a node set V, a set $L \subseteq V$ of admissible leaves, and a set J of justifications. A justification j has a premise set A_j, a conclusion c_j, and a certification predicate. Define its support by

$$S_j = A_j \cup \{c_j\}.$$

A set $X \subseteq V$ is a *model of the declared field* when $L \subseteq X$ and, for every certified j,

$$A_j \subseteq X \quad \Longrightarrow \quad c_j \in X.$$

Each justification is one implication with conjunctive premises. Alternative justifications remain separate implications. Uncertified attempts and failed-route annotations do not contribute a deduction rule merely by appearing in the archive.

Let Der_F be the inductively generated set obtained from the leaves by certified rules. For a finite field, repeated forward closure computes this set. It is not necessary to enumerate all valuations to perform that computation.

Theorem 24.1 (Soundness and relative completeness)**.** *For every node v,*

$$v \in \mathrm{Der}_F \quad \Longleftrightarrow \quad v \in X \text{ for every model } X \text{ of the declared field.}$$

If v is not derivable, Der_F itself is a model omitting v.

Proof. Every model contains the leaves. Induction on a derivation shows that it contains the conclusion of every certified application whose premises have already been derived. Thus every model contains Der_F. Conversely, the inductive constructors show that Der_F contains the leaves and is closed under every certified rule. It is therefore a model. Applying the universal condition to this particular model proves the reverse implication, and the same model gives the stated countermodel. □

This is relative completeness for the declared rule system, not completeness of arbitrary mathematical proof search. A missing theorem may simply require a rule or a lemma absent from the field. A failed search can therefore produce a useful countermodel to a proposed finite inference architecture without disproving the mathematical target.

For an intended interpretation $\mu : V \to \mathrm{Prop}$, the transfer to mathematical truth has explicit hypotheses: every leaf must satisfy μ, and every certified rule must preserve μ from all its premises to its conclusion. Under those hypotheses, induction gives $v \in \mathrm{Der}_F \Rightarrow \mu(v)$. Lean checks this statement as `ProofPath.Field.semantic_soundness`. It does not infer those hypotheses from a JSON certification flag.

24.2 From patches to a contravariant view system

For a patch $U \subseteq V$, let $\mathcal{M}_F(U)$ consist of subsets $X_U \subseteq U$ which contain $L \cap U$ and obey every certified rule whose entire support lies in U. If $U \subseteq W$, define

$$\rho_{U,W} : \mathcal{M}_F(W) \longrightarrow \mathcal{M}_F(U), \qquad \rho_{U,W}(X_W) = X_W \cap U.$$

This map is well defined. A rule supported in U is also supported in W, and its conclusion remains in U. The restriction therefore preserves both the local leaves and the local implication conditions.

Proposition 24.2 (The proof-field view laws)**.** *The restriction maps satisfy*

$$\rho_{U,U} = \mathrm{id}, \qquad \rho_{U,W}\rho_{W,Z} = \rho_{U,Z} \quad (U \subseteq W \subseteq Z).$$

Consequently $U \mapsto \mathcal{M}_F(U)$ is a contravariant set-valued view system on the inclusion order of patches.

Proof. Both laws are the corresponding intersection identities. For the second, $(X \cap W) \cap U = X \cap U$ because $U \subseteq W$. The values outside a patch are absent, or equivalently normalized to false in the Lean representation. There are no additional hidden coordinates. □

The strict Komorebi realization is now an actual set of coherent local models,

$$\Gamma(\mathcal{M}_F) = \{(X_U)_U : \rho_{U,W}(X_W) = X_U \text{ whenever } U \subseteq W\}.$$

There is an explicit bijection

$$\mathrm{Models}(F) \simeq \Gamma(\mathcal{M}_F), \qquad X \longmapsto (X \cap U)_U.$$

Its inverse evaluates a coherent family at the full patch V. Coherence shows that every other component is precisely the restriction of that value. The Lean declaration `ProofPath.Field.modelEquivSections` constructs this equivalence, using `ViewDiagram` and the actual local restriction maps.

The full-patch equivalence identifies the ambient construction. It does not prove that arbitrary prescribed proper-patch data extend: the full patch already contains the global problem. Extension from smaller views is the distinct content of the following theorem.

24.3 The correct coverage condition is a whole support

Let $(U_i)_{i \in I}$ cover the nodes. A family $X_i \in \mathcal{M}_F(U_i)$ is matching when

$$X_i \cap U_j = X_j \cap U_i \qquad \text{for every } i, j.$$

This equality includes agreement about false values, not merely common true values.

Theorem 24.3 (Whole-support gluing)**.** *Suppose every certified justification has its whole support in some patch:*

$$\forall j \text{ certified}, \quad \exists i \quad S_j \subseteq U_i.$$

Then every matching family (X_i) has a unique global field model X satisfying $X \cap U_i = X_i$ for all i.

Proof. Set $X = \bigcup_i X_i$. Matching and node coverage imply $X \cap U_i = X_i$. A leaf belongs to some patch and hence to its local model, so $L \subseteq X$. Consider a certified rule whose premises belong to X. Choose a patch containing its entire support. Matching implies that every premise is true in that patch's local model, whose rule condition forces the conclusion. Hence X obeys all certified rules. Any other global model with the specified restrictions agrees with X at every node because the patches cover the nodes. This proves uniqueness. □

Example 24.4 (Pairwise dependencies do not suffice)**.** Take nodes a, b, c, no required leaves, and one certified rule

$$\{a, b\} \longrightarrow c.$$

Use patches $U_0 = \{a, c\}$ and $U_1 = \{b, c\}$. The local valuations $X_0 = \{a\}$ and $X_1 = \{b\}$ agree on their overlap: both assign false to c. Both are local models, since neither patch contains the full rule support. They cover every node and both pairwise premise-to-conclusion dependencies. Nevertheless, a global model realizing them would contain a, b and omit c, contradicting the rule. Thus replacing a conjunctive hyperedge by unrelated pairwise checks can lose precisely the constraint that matters.

The example is kernel checked in `ProofPath.Examples`. It does not contradict the earlier arrow-cover theorem. That theorem applies to a problem already specified by binary restriction equations. A conjunctive proof rule is a different constraint until a correct representation of its whole support has been supplied. The full support may be redundant in particular fields, so the theorem gives a sufficient cover condition rather than a necessary one for every individual diagram.

The Python class `ProofFieldViews` implements these definitions. It freezes the rule and leaf contents at construction, restricts local valuations by literal intersection, checks support coverage before gluing, and can form a finite intersection-closed diagram for `FiniteViewSystem`. Local enumeration has explicit size limits. Forward closure and declared-field countermodels do not require that enumeration.

24.4 Protected realization over a field

The exact binary model admits a general algebraic formulation. Let K be a field, let E, R, S be K-vector spaces, and let $D : E \to R$ and $p : E \to S$ be linear. The prescribed problem is to repair x while retaining $p(x)$:

$$Dy = 0, \qquad p(y) = p(x).$$

The admissible correction image is $B = D(\ker p)$. The elementary correction equivalence gives feasibility exactly when $Dx \in B$.

Theorem 24.5 (A linear certificate alternative)**.** *The protected problem is infeasible if and only if there is a linear functional $\lambda : R \to K$ satisfying*

$$\lambda(Dh) = 0 \quad \textit{whenever } p(h) = 0, \qquad \lambda(Dx) = 1.$$

For a finite matrix M and right side b, the corresponding alternative is

$$\nexists y \ (My = b) \quad \Longleftrightarrow \quad \exists \ell \ (\ell M = 0 \textit{ and } \ell b = 1).$$

Proof. If $Dx \notin B$, linear separation of a vector from a subspace gives a functional vanishing on B and nonzero at Dx. One construction extends a basis of B by Dx and then to a basis of R. Assign value zero to the basis of B and value one to Dx, and extend linearly. Equivalently, normalize any separating

functional by its nonzero value. Conversely, if y were feasible, then $h = x - y$ would satisfy $p(h) = 0$ and $Dh = Dx$, contradicting the displayed functional equations. For a matrix, take B to be its column image. In finite coordinates the separating functional is a row vector, giving the stated alternative. □

The general existence proof is classical. It does not claim an efficient procedure for arbitrary infinite spaces. In the implemented binary case, Gaussian elimination constructs a row witness, and the witness is checked against the original matrix and right side. A positive solution and a normalized negative witness cannot both verify.

24.5 Counting what remains after an observation

Let $T : E \to R$ and $Q : E \to W$ be linear, and fix a known point x_0. Put

$$\mathcal{A} = \{y : Ty = Tx_0\}, \quad H = \ker T, \quad Q_H = Q|_H.$$

Translation $h \mapsto x_0 + h$ identifies H with $\mathcal{A}$. The observation image is the translate $Qx_0 + Q(H)$, and the fiber above the attained value Qx_0 is a translate of $\ker Q_H$.

Theorem 24.6 (Exact observation counts)**.** *Suppose K has q elements and E is finite. Set $k = \dim_K H$ and $r = \dim_K Q(H)$. Then*

$$|\mathcal{A}| = q^k, \qquad |Q(\mathcal{A})| = q^r, \qquad |\{y \in \mathcal{A} : Qy = Qx_0\}| = q^{k-r}.$$

In particular the exact count identity is

$$|\mathcal{A}| = |\{y \in \mathcal{A} : Qy = Qx_0\}|\, q^r.$$

Every attained observation fiber has the same size. For the uniform distribution on $\mathcal{A}$, the observation removes exactly $r \log_2 q$ bits of uncertainty.

Proof. The two translation equivalences reduce the counts to finite vector spaces. Such a space of dimension d has q^d elements. Rank-nullity for Q_H gives $\dim \ker Q_H + r = k$, proving the product identity and all the counts. An attained value can be treated by choosing a point attaining it as the base point. Equal fiber sizes imply a uniform observation distribution and a uniform conditional distribution on each fiber. The entropy difference is therefore $\log_2(q^k) - \log_2(q^{k-r}) = r \log_2 q$. □

Lean checks the translation equivalences, rank-nullity specialization, fiber cardinalities, and product identity in `ProofPath.Protected`, with the binary specialization in `ProofPath.Binary`. The logarithmic entropy interpretation is the written corollary above, not a newly compiled theorem about a Lean probability distribution. Uniformity is substantive. With a nonuniform prior, dimension alone does not determine Shannon information.

For the protected problem, set $T = (D, p)$ and choose an actual feasible y_0, so $Ty_0 = (0, px)$. This reduces its solution fiber to the theorem without changing the protected values. A concrete binary example has $a + b + c = 0$ and $a = 0$. Its two realizations are $(0, 0, 0)$ and $(0, 1, 1)$. Observing b has rank one on the ambiguity space and determines the remaining coordinate. Repeating the same observation in two output rows does not double the information.

24.6 Executable witnesses and a native declaration export

Four exact examples are regenerated from the Python implementation: a one-bit ambiguity, an incompatible protected assignment, a redundant observation, and a uniquely determined realization. The generator emits the original stacked matrices and right sides alongside the proposed solutions or row witnesses. It also emits Lean declarations which check those concrete equations and cardinalities by kernel reduction. A faulty generator cannot make a false displayed proposition pass merely by assigning a certification flag. The generator itself is not thereby verified for arbitrary inputs, and the correspondence between the supplied JSON input and its emitted Lean syntax remains an audited software boundary.

A separate native exporter reads five already compiled Lean declarations, extracts their direct type and proof-constant dependencies, and records the transitive axiom sets. Dependencies outside the selected roots are explicit external leaves. This yields an actual proof-field JSON record rather than a hand-authored imitation of one. The exported field is then loaded through the public Python importer, and its declared closure is checked to contain its roots.

The export has a deliberately narrow meaning. It is syntactic dependency data for selected declarations, not a reconstruction of the entire discovery process. It does not recover failed attempts, prove that every constant in a type was a historically essential idea, or formally verify the metaprogram that extracts the data. The mathematical proof-field bridge is checked independently of that metaprogram. Its semantic interpretation retains the explicit hypotheses in Theorem 24.1.

24.7 Verification scope and reproducibility

The release contains eight production Lean modules. Each is rebuilt from source into a fresh object directory using the pinned compiler and dependency revisions. The project is then replayed through Lean's own kernel against its imported environment, and every selected named declaration receives an axiom audit. This is not an independent implementation of Lean, and the cached Mathlib library is not rebuilt or replayed in its entirety.

Four deliberate invalid files test rejection of a false equality, an invalid row witness, the invalid conjunction model, and a proof placeholder. A fifth file introduces an explicit false axiom. Lean correctly accepts a consequence relative to that axiom, while the release's separate axiom policy rejects the dependency. A missing import or a timeout does not count as a successful mathematical rejection.

The same verification run exercises the software tests, inherited exact comparisons, regenerated examples, native export, package installation, and manuscript build. Machine-readable logs distinguish written proofs, general Lean theorems, concrete checked examples, and implementation tests. The release does not use one category as a substitute for another.

25 Shared interpretation, review dependence, and identification

The distinction between proof and proof field also matters when many people assess the same mathematics. An individual can evaluate a theorem through the notation, examples, expectations, and disciplinary assumptions available to them. Different reviewers may share these influences. Agreement therefore need not provide independent evidence, although substantive criticism can still identify a real defect. The mathematical contribution of a model lies in making its assumptions explicit, not in

diagnosing the motives of reviewers.

25.1 Correlated noise and persistent common bias

The covariance model of Section 17 measures how dependence reduces the effective information in repeated estimates. It cannot remove a common offset. The next proposition puts both effects into one equation.

Proposition 25.1 (MSE with a shared interpretive offset)**.** *Suppose reviewers report $J_i = \theta + b + \varepsilon_i$, where θ is the target quantity, b is a fixed shared offset, $\mathbb{E}\varepsilon_i = 0$, $\mathrm{Var}(\varepsilon_i) = \sigma^2$, and $\mathrm{Cov}(\varepsilon_i, \varepsilon_j) = \rho\sigma^2$ for $i \neq j$. For admissible ρ, the mean squared error of the reviewer average is*

$$\boxed{\mathbb{E}(\overline{J}_n - \theta)^2 = b^2 + \sigma^2\left(\rho + \frac{1-\rho}{n}\right).}$$

For fixed $\rho \geq 0$, it approaches $b^2 + \rho\sigma^2$ as $n \to \infty$.

Proof. The estimation error is $\overline{J}_n - \theta = b + \overline{\varepsilon}_n$. Its mean is b, and its variance is $\sigma^2\big((1-\rho)/n + \rho\big)$ by summing all diagonal and off-diagonal covariance entries. Mean squared error is variance plus squared bias. Taking the limit completes the proof. □

This formula complements, rather than replaces, the effective reviewer count. Adding reviewers may reduce independent error while leaving the common offset untouched. The existing Python routine `shared_bias_mse` computes this model and permits an additional declared variance term for a shared random lens. Its input parameters are model choices, not measured facts about a given intellectual community.

25.2 When an identity effect can actually be measured

It is easy to speculate that authorship or disciplinary identity changed an evaluation; it is harder to establish such an effect. A direct experimental design can hold an argument fixed while randomizing its surrounding label. Let $Y_i(1)$ and $Y_i(0)$ denote the evaluations that a reviewer would give the same declared mathematical content under two different labels. In a randomized design, the difference of treatment-group and control-group mean responses estimates the population average label effect, subject to the usual sampling, assignment, and interference conditions. It does not establish whether the proof is correct. That requires a separate audit of the mathematical claim.

This distinction is central to a proposed application of Proof Path Theory. One may observe that a presentation changes which branches reviewers explore or how they estimate importance. Those differences become evidence of a contextual effect only after alternative explanations have been addressed. A skeptical review can be correct. Agreement can be justified. The mere presence of community influence neither validates nor invalidates a theorem.

The research opportunity is to connect a typed archive of mathematical claims with controlled studies of how proof paths are selected, compressed, and interpreted. A certified proof core would be held fixed while the accompanying discovery field or contextual labels change. The questions are which alternatives readers recognize, which obstructions they find, and how their subsequent mathematical exploration differs. These are empirical targets for future work, not conclusions established by this finite model.

25.3 What the new bridge does and does not claim

The new sections join three previously separated themes. The ambient field retains unsuccessful and unresolved mathematical trajectories. Komorebi Space determines when selected local realizations are coherent and whether prescribed information extends. A quantitative review model distinguishes added headcount from independent information. Together they furnish a sharper architecture for machine-supported mathematical inquiry.

These constructions do not yield a general solver for mathematical conjectures, a unique numerical measure of mathematical meaning, a canonical global proof field, or an empirical diagnosis of actual mathematical communities. They provide explicit finite questions with checkable answers and identify the conditions needed for stronger generalizations. Most importantly, they preserve the difference between a valid result and the circumstances through which people discover or evaluate it.

26 What the theory establishes

The completed finite theory establishes the following structural facts.

1. A proof field can represent conjunction within a justification and alternatives among justifications without confusing candidate edges with certified edges.
2. Derivability in a finite acyclic selected subfield is well defined and computable in topological order.
3. A fixed AND-DAG has a unique minimal verification core.
4. Alternative derivations make minimum certificate selection NP-complete in general.
5. Compression is meaningful only relative to a task family.
6. Task-relative recoverability forms a preorder and a quotient partial order.
7. Verification loss, discovery loss, meaning loss, bridge loss, and audit loss can diverge.
8. Expected reconstruction debt decreases monotonically as more required material is retained.
9. Bridge criticality is domination in the declared future-reach graph.
10. Weighted future reach is monotone and submodular.
11. Greedy cardinality retention has the standard $(1 - 1/e)$ guarantee.
12. Append-only certified updates preserve existing derivability.
13. The forward dependency closure is the smallest universally sound syntactic invalidation set.
14. Review dependence reduces effective independent review, and generalized least squares gives the optimal linear unbiased aggregate under a known covariance model.
15. Constant-rate audit queues have an exact stable, critical, or unstable classification.
16. Optimal proof expansion depth depends on the presentation goal.

These results turn the original methodological insight into a finite mathematical branch with explicit objects, algorithms, complexity, and failure boundaries.

The ambient extensions add a finite view system, scope-aware failed-path analysis, the protected linear realization alternative, exact observation gain, and an explicit interpretation model. Version 1.2 gives

the proof field an actual contravariant local-model diagram and a checked equivalence with its coherent sections. Whole-support gluing establishes extension from suitable proper patches. The native exporter reads compiled Lean declarations, but arbitrary semantic interpretation and general correctness of the exporter are not inferred from that successful execution.

27 Limitations and claim boundaries

Proof Path Theory does not establish a theorem by preserving the history of an argument. A richly documented wrong proof remains wrong. Bridge value is not validity. Explanatory power is not validity. Consensus is not validity. The certified subfield remains the object that carries deductive force.

The dependency graph is a declared model. It records the dependencies that were formalized, not every historical influence on discovery. Tacit knowledge, visual intuition, unrecorded conversations, and changes in vocabulary may be absent.

The value and cost coordinates are task models. Verification can often be specified sharply. Meaning and future bridge value are partly prospective. Their use is justified when assumptions are stated, not when the numbers are treated as intrinsic physical constants.

The finite bridge graph does not predict every future theorem. It supports conditional statements of the form: under these declared future targets and transfer edges, this retention policy preserves this amount of modeled reach.

The exact minimum certificate algorithm is exponential in the worst case. The package makes this limitation explicit through a search-state bound.

The review and queue models are deliberately elementary. They isolate dependence and capacity effects, but they do not model every institutional or psychological feature of mathematical review.

Finally, the paper does not claim that graphs, hypergraphs, proof theory, submodular optimization, covariance-aware aggregation, or queueing recurrences were invented here. The contribution is the formal assembly of these structures around proof compression, discovery fields, mathematical memory, and versioned proof portfolios.

28 Open theorem program

Problem 28.1 (Proof-path view functors and comparison)**.** The declared-field local-model functor and whole-support gluing are established here. Extend the checked interface to richer proof-state representations, and verify the implementation of general proof assistant adapters. Determine when objects with symmetry require nonabelian or higher descent rather than the strict set-valued construction.

Problem 28.2 (Counterfactual proof-search observations)**.** Study which observations on a finite realization fiber yield the greatest task-relative reduction of uncertainty per unit cost. Extend the binary rank theorem to controlled infinite or nonlinear settings only after specifying distributions, admissible questions, and the semantics of proof state.

Problem 28.3 (Collective evaluation without shared-context confounding)**.** Design preregistered evaluations of identical certified mathematics under randomized labels and alternative explanatory contexts. Separate measurement of interpretation differences from independent verification of correctness, and test whether models of correlated review predict actual audit errors.

The finite theory leaves several focused directions.

Problem 28.4 (Infinite proof fields)**.** Develop locally finite and countable proof fields with conditions guaranteeing that finite certificates approximate an infinite derivation without losing domain or convergence information.

Problem 28.5 (Parameterized certificate complexity)**.** Classify minimum certificate selection by treewidth, number of alternative justifications, justification arity, shared-subproof width, and target count. Identify fixed-parameter tractable regimes and approximation bounds.

Problem 28.6 (Probabilistic bridge calibration)**.** Replace declared bridge weights by calibrated posterior models while keeping conjectural reach separate from certified derivability.

Problem 28.7 (Proof assistant interoperability)**.** Construct adapters that export stable claim identifiers, primitive dependencies, scopes, and proof terms from existing formal systems into the common proof-field schema.

Problem 28.8 (Higher proof transformations)**.** Model transformations between proofs, transformations between transformations, and coherent compiler pipelines in a higher categorical setting.

Problem 28.9 (Privacy and attribution)**.** Design proof archives that preserve provenance and priority while allowing private notes, embargoed routes, and sensitive collaboration records to remain access controlled.

Problem 28.10 (Empirical validation)**.** Study real mathematical projects to estimate recovery costs, audit bottlenecks, and the predictive value of bridge-retention scores. Such work should test the models rather than assume them.

Problem 28.11 (Distributed audit systems)**.** Extend the queue model to heterogeneous reviewers, dependent tasks, rework, priority classes, and claims whose audit cost changes after neighboring claims are resolved.

29 Conclusion

The version 1.2 continuation sharpens the original distinction between a proof certificate and the larger proof field. The field need not consist only of viable derivations. It can retain failed strategies and genuine local refutations without allowing either to masquerade as global unprovability. A checked contravariant system of local field models gives an explicit bridge to Komorebi Space. The need to cover whole justifications identifies a constraint that a pairwise dependency graph can miss. In the linear finite case, protected observations admit exact success or failure certificates and a measurable information gain. A separate review model distinguishes proof correctness from correlated social interpretation.

A final proof is indispensable, but it is not always the whole mathematical object. The completed theory distinguishes the certificate from the proof field that produced it. It gives that field typed nodes, alternative justifications, evidence records, task-relative compressions, bridge graphs, version histories, and explicit audit models.

The main mathematical boundary is now clear. Verification is a closure property of the certified subfield. Discovery value belongs to a larger graph. Compression can preserve the first while reducing the second. Once that separation is formal, several consequences follow. Minimal certificates can be extracted exactly in fixed AND-DAGs. Alternative derivations create a genuine combinatorial optimization

problem. Future-reach retention becomes submodular coverage. Version changes propagate through descendants. Correlated review has an effective sample size. Candidate generation is constrained by audit capacity. Expansion depth depends on purpose.

The final thesis is therefore:

> A proof should be published as a certificate and maintained as a versioned proof field. The certificate preserves deductive validity. The larger field preserves the dependencies, alternatives, examples, failures, provenance, and bridges from which later mathematics may grow.

A Theorem ledger

Proved denotes a written proof. *Lean* denotes the specified checked interface, not a formalization of the entire paper. *Mixed* distinguishes checked counts from their written probabilistic interpretation. The separate Lean concordance lists every source declaration and its scope.

ID	Result	Status	Executable support
PPT-001	Finite derivability is unique and computable in topological order.	Proved	`ProofField.derive`
PPT-002	An AND-DAG has a unique minimal verification core.	Proved	`unique_and_dag_certificate`
PPT-003	AND-DAG core extraction is linear with indexed adjacency.	Proved	graph algorithm, not a runtime certificate
PPT-004	Minimum Certificate Decision is NP-complete.	Proved	Set Cover example
PPT-005	Certified paths transport functorially.	Proved	structural theorem
PPT-006	Local auditable certificates compose under a certificate functor.	Proved	structural theorem
PPT-007	Task information loss follows from compression collisions.	Proved	task API
PPT-008	Recoverability is a preorder.	Proved	`recoverability_preorder`
PPT-009	The quotient by mutual recoverability is a partial order.	Proved	equivalence API
PPT-010	Minimal sufficient compression depends on the declared task family.	Proved	finite countermodel
PPT-011	Zero verification loss can coexist with positive discovery loss.	Proved	AND-DAG example
PPT-012	Compression debt is monotone under retention.	Proved	`compression_debt`
PPT-013	Bridge criticality equals target domination.	Proved	`dominated_targets`
PPT-014	Weighted future reach is monotone and submodular.	Proved	exhaustive finite checker
PPT-015	Greedy cardinality retention achieves at least $1 - 1/e$.	Proved	greedy and exact solvers

ID	Result	Status	Executable support
PPT-016	Append-only certified updates preserve derivability.	Proved	versioning check
PPT-017	Forward dependency closure is the minimal closed syntactic review set.	Proved	`affected_descendants`
PPT-018	Equicorrelated review has the stated effective count.	Proved	covariance implementation
PPT-019	GLS weights minimize unbiased linear review variance.	Proved	`generalized_least_squares_weights`
PPT-020	Constant-rate audit queues have exact stable, critical, and unstable regimes.	Proved	queue simulation
PPT-021	Optimal expansion depth depends on the goal.	Proved	refinement implementation
PPT-022	Pure retention projections commute and are idempotent.	Proved	`retention_projection`
PPT-023	Retention masks form a Boolean lattice.	Proved	`retention_meet`, `retention_join`
PPT-024	A locally refuted route can coexist with another certified route.	Proved	Alternative justification field
PPT-025	A checked local refutation persists under unchanged interpretation.	Proved	Scoped witness principle
PPT-026	Finite arrow-covered matching views glue uniquely.	Lean	`FiniteViewSystem.glue`
PPT-027	Protected realizations admit exact solutions or obstruction witnesses, with classified fibers.	Lean	`analyze_affine_realizations`
PPT-028	Finite affine fibers have exact observation counts. Uniform entropy is a written corollary.	Mixed	Lean counts and binary tests
PPT-029	Common offset and correlated error have the displayed reviewer MSE.	Proved	`shared_bias_mse`
PPT-030	Declared derivability equals truth in every declared field model.	Lean	Closure and countermodels
PPT-031	Local models form a contravariant diagram with an explicit realization equivalence.	Lean	`Field.modelEquivSections`
PPT-032	Whole-support matching local models glue; pairwise coverage can fail.	Lean	Proof-field views and counterexample
PPT-033	All predicate tasks are sufficient exactly for injective encodings.	Lean	Compression collision theorem

B Reproduction

From the extracted section root, with the separate pinned offline runtime available:

```
python -m pip install -e . --no-build-isolation
python scripts/verify_release.py --runtime /path/to/portable-kit
# Only the formal bridge:
```

```
python scripts/verify_lean.py --runtime /path/to/portable-kit
# Finite computations without a Lean claim:
python -m pytest
python examples/ambient_realization.py
# Standalone paper build uses docs/ as its working directory:
cd docs
pdflatex -interaction=nonstopmode -halt-on-error Proof_Path_Theory.tex
pdflatex -interaction=nonstopmode -halt-on-error Proof_Path_Theory.tex
pdflatex -interaction=nonstopmode -halt-on-error Proof_Path_Theory.tex
```

The source archive also contains a wheel build script and a GitHub Actions workflow fragment for a multi-section repository.

C Canonical proof-field schema excerpt

```
{
  "schema_version": "1.0",
  "premises": ["A"],
  "targets": ["B"],
  "nodes": [
    {
      "id": "A",
      "label": "Premise A",
      "kind": "premise",
      "values": {"verification": 1.0},
      "costs": {"storage": 0.2}
    }
  ],
  "evidence": [
    {
      "id": "e-proof",
      "kind": "formal_proof",
      "source": "declared derivation",
      "scope": "",
      "certifies": true
    }
  ],
  "justifications": [
    {
      "id": "j1",
      "premises": ["A"],
      "conclusion": "B",
      "rule": "deduction",
      "certified": true,
      "evidence_ids": ["e-proof"]
    }
  ]
}
```

References

[1] Komorebi P. *Komorebi Space: Coherent Realizations, Obstructions, and Descent*. Monograph, 2026. The finite proof-field view construction here is a relative application of its strict coherent realization principle, not a claim of unrestricted semantic completeness.

[2] S. R. Buss, editor. *Handbook of Proof Theory*. Elsevier, 1998.

[3] T. M. Cover and J. A. Thomas. *Elements of Information Theory*. Second edition, Wiley, 2006.

[4] R. Diestel. *Graph Theory*. Fifth edition, Springer, 2017.

[5] M. R. Garey and D. S. Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman, 1979.

[6] D. E. Knuth. *Literate Programming*. CSLI Publications, 1992.

[7] S. Mac Lane. *Categories for the Working Mathematician*. Second edition, Springer, 1998.

[8] G. L. Nemhauser, L. A. Wolsey, and M. L. Fisher. An analysis of approximations for maximizing submodular set functions I. *Mathematical Programming*, 14:265–294, 1978.

[9] A. S. Troelstra and H. Schwichtenberg. *Basic Proof Theory*. Second edition, Cambridge University Press, 2000.